Introduction aux réseaux euclidiens

Plan

Teasing : sécurité de RSA avec Petits exposants secrets

Introduction aux réseaux euclidiens

Propriétés de base, plus court vecteur, volume

Problèmes algorithmiques dans les réseaux

Algorithmes de « réduction de réseau »
- En dimension 2 : réduction de Lagrange
- Algorithmes basés sur l'orthogonalisation

Plan

Teasing : sécurité de RSA avec Petits exposants secrets

Introduction aux réseaux euclidiens

Propriétés de base, plus court vecteur, volume

Problèmes algorithmiques dans les réseaux

Algorithmes de « réduction de réseau »
- En dimension 2 : réduction de Lagrange
- Algorithmes basés sur l'orthogonalisation
  - Orthogonalisation de Gram-Schmidt
  - La size-reduction

Signature RSA

Génération de clef classique

1. Choisir $p, q \approx 2^{n/2}$ aléatoires, premiers.
2. Calculer $N = pq$ et $\phi(N) = (p-1)(q-1)$
3. Choisir $e = 3$
4. Si e n'est pas inversible modulo $\phi(N)$, retourner en 1.
5. Calculer $d \leftarrow e^{-1} \bmod \phi(N)$

À la fin, $ed \equiv 1 \bmod \phi(N)$

Bilan : $N \approx 2^n$, $e = 3$, $d \approx 2^n$.

- Signature = $2n$ multiplications $\bmod N \rightsquigarrow \mathcal{O}\left(n^3\right)$.
- Vérification = 2 multiplications $\bmod N \rightsquigarrow \mathcal{O}\left(n^2\right)$.

Signer est plus lent que vérifier

La CB signe, le terminal vérifie... dommage !
Peut-on faire l'inverse ?

Signature RSA

Génération de clef avec petit exposant secret

1. Choisir $p, q \approx 2^{n/2}$ aléatoires, premiers
2. Calculer $N = pq$ et $\phi(N) = (p-1)(q-1)$
3. Choisir $\cancel{d = 3}$ $d \approx 2^{128}$ aléatoire
4. Si d n'est pas inversible modulo $\phi(N)$, retourner en 1.
5. Calculer $e \leftarrow d^{-1} \bmod \phi(N)$ À la fin, $ed \equiv 1 \bmod \phi(N)$

Bilan : $N \approx 2^n$, $e \approx 2^n$, $d \approx 2^{128}$.

- Signature = 256 multiplications $\bmod N \rightsquigarrow \mathcal{O}\left(n^2\right)$.
- Vérification = $2n$ multiplications $\bmod N \rightsquigarrow \mathcal{O}\left(n^3\right)$.

C'est 16× plus rapide, mais...

Est-ce sûr ?

Modélisation du problème

- On connaît e et N
- BUT : retrouver d

On sait que :

$$ed = 1 + k\phi(N)$$

$$0 \leq d \leq N^{\alpha} \qquad \alpha = \frac{1}{16}$$

$$0 \leq k \leq N^{\alpha} \qquad k = (ed - 1)/\phi(N) < d$$

Modélisation du problème

- On connaît e et N
- BUT : retrouver d

On sait que :

$$ed = 1 + k(N - \sigma + 1) \qquad \sigma = p + q$$

$$0 \leq d \leq N^{\alpha} \qquad \alpha = \frac{1}{16}$$

$$0 \leq k \leq N^{\alpha} \qquad k = (ed - 1)/\phi(N) < d$$

$$0 \leq \sigma \leq 2N^{\frac{1}{2}}$$

Modélisation du problème

- On connaît e et N
- BUT : retrouver d

On sait que :

$$ed = 1 + k(N+1) - k\sigma \qquad \sigma = p + q$$

$$0 \leq d \leq N^{\alpha} \qquad \alpha = \frac{1}{16}$$

$$0 \leq k \leq N^{\alpha} \qquad k = (ed - 1)/\phi(N) < d$$

$$0 \leq \sigma \leq 2N^{\frac{1}{2}}$$

Modélisation du problème

- On connaît e et N
- BUT : retrouver d

On sait que :

$$ed = k(N+1) - y \qquad \sigma = p + q, y = k\sigma - 1$$

$$0 \leq d \leq N^{\alpha} \qquad \alpha = \frac{1}{16}$$

$$0 \leq k \leq N^{\alpha} \qquad k = (ed - 1)/\phi(N) < d$$

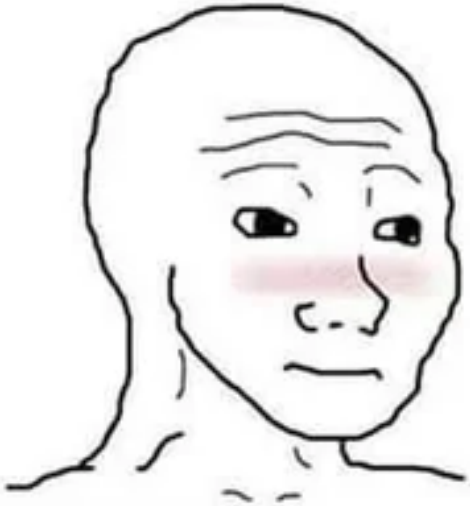
$$0 \leq y \leq 2N^{\frac{1}{2}+\alpha}$$

Technique générale : linéarisation

Introduire $y = k\sigma - 1$ pour éliminer le terme non-linéaire.

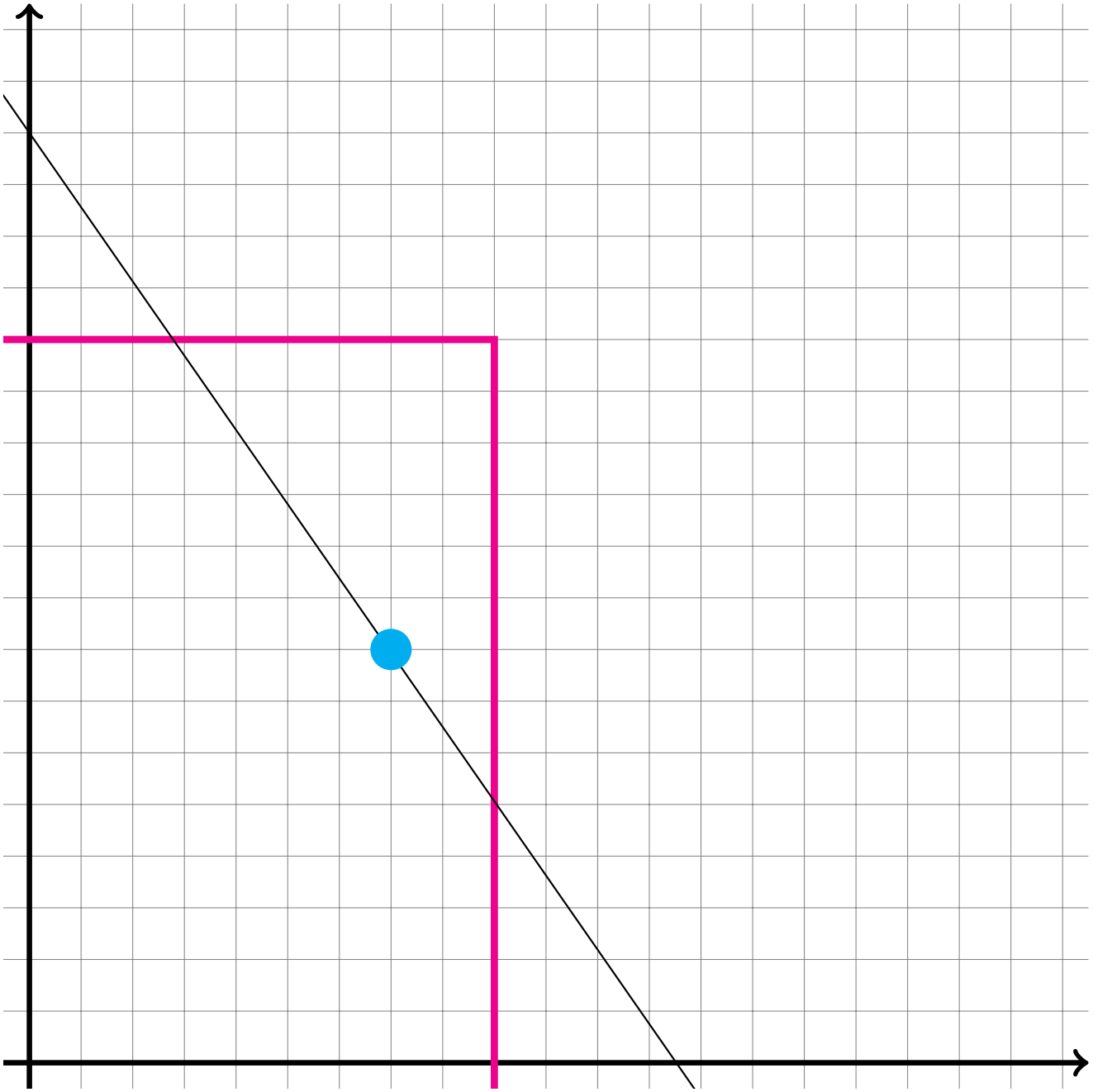
The world is a cruel and unjust place. There is no harmony in the universe. The only constant is suffering.

minutes later



OMG, I can linearize this problem

Géométriquement



Programmation linéaire

« Programme linéaire »

$$\begin{cases} ed - k(N+1) + y = 0 \\ 0 \leq d \leq N^{\alpha} \\ 0 \leq k \leq N^{\alpha} \\ 0 \leq y \leq 2N^{\frac{1}{2}+\alpha} \end{cases}$$

- La programmation linéaire sur les flottants est **polynomiale**
  - Khachiyan, 1979 – URSS
- La programmation linéaire sur les entiers est **NP-dure**
- Algorithmes polynomiaux si #variables est constant
  - exponentiel en #variables, polynomial en |coefficients|
  - H. Lenstra, 1981
  - Basé sur les réseaux euclidiens

Programmation linéaire

« Programme linéaire »

$$\begin{cases} 0 \leq k(N+1) - ed \leq 2N^{\frac{1}{2}+\alpha} \\ 0 \leq d \leq N^{\alpha} \\ 0 \leq k \leq N^{\alpha} \end{cases}$$

- La programmation linéaire sur les flottants est **polynomiale**
  - Khachiyan, 1979 – URSS
- La programmation linéaire sur les entiers est **NP-dure**
- Algorithmes polynomiaux si #variables est constant
  - exponentiel en #variables, polynomial en |coefficients|
  - H. Lenstra, 1981
  - Basé sur les réseaux euclidiens

Plan

Teasing : sécurité de RSA avec Petits exposants secrets

Introduction aux réseaux euclidiens

Propriétés de base, plus court vecteur, volume

Problèmes algorithmiques dans les réseaux

Algorithmes de « réduction de réseau »
- En dimension 2 : réduction de Lagrange
- Algorithmes basés sur l'orthogonalisation
  - Orthogonalisation de Gram-Schmidt
  - La size-reduction

Rappels

- Vecteur $\mathbf{x} = (x_1, \ldots, x_n)$.
- Norme (euclidienne) (« norme 2 », « longueur ») :

$$\|\mathbf{x}\| = \sqrt{\mathbf{x}, \mathbf{x}\rangle} = \sqrt{{x_1}^2 + {x_2}^2 + \cdots + {x_n}^2}$$

$\mathbb{R}$-Espace vectoriel engendré par $\mathbf{b}_1, \ldots, \mathbf{b}_m$

- Ensemble des combinaisons linéaires de $\mathbf{b}_1, \ldots, \mathbf{b}_m$

$$\mathrm{Span}(\mathbf{b}_1, \ldots, \mathbf{b}_m) = \left\{ \sum_{i=1}^{m} \lambda_i \mathbf{b}_i : (\lambda_1, \ldots, \lambda_m) \in \mathbb{R}^m \right\}$$

- $\mathbf{x}, \mathbf{y} \in V \Longrightarrow \lambda\mathbf{x} + \mu\mathbf{y}$ pour tout $\lambda, \mu \in \mathbb{R}$

Réseau euclidien

Un réseau euclidien (« *Euclidean lattice* »), c'est la même chose, mais avec des entiers. Il suffit de remplacer $\mathbb{R}$ par $\mathbb{Z}$ dans la diapo précédente.

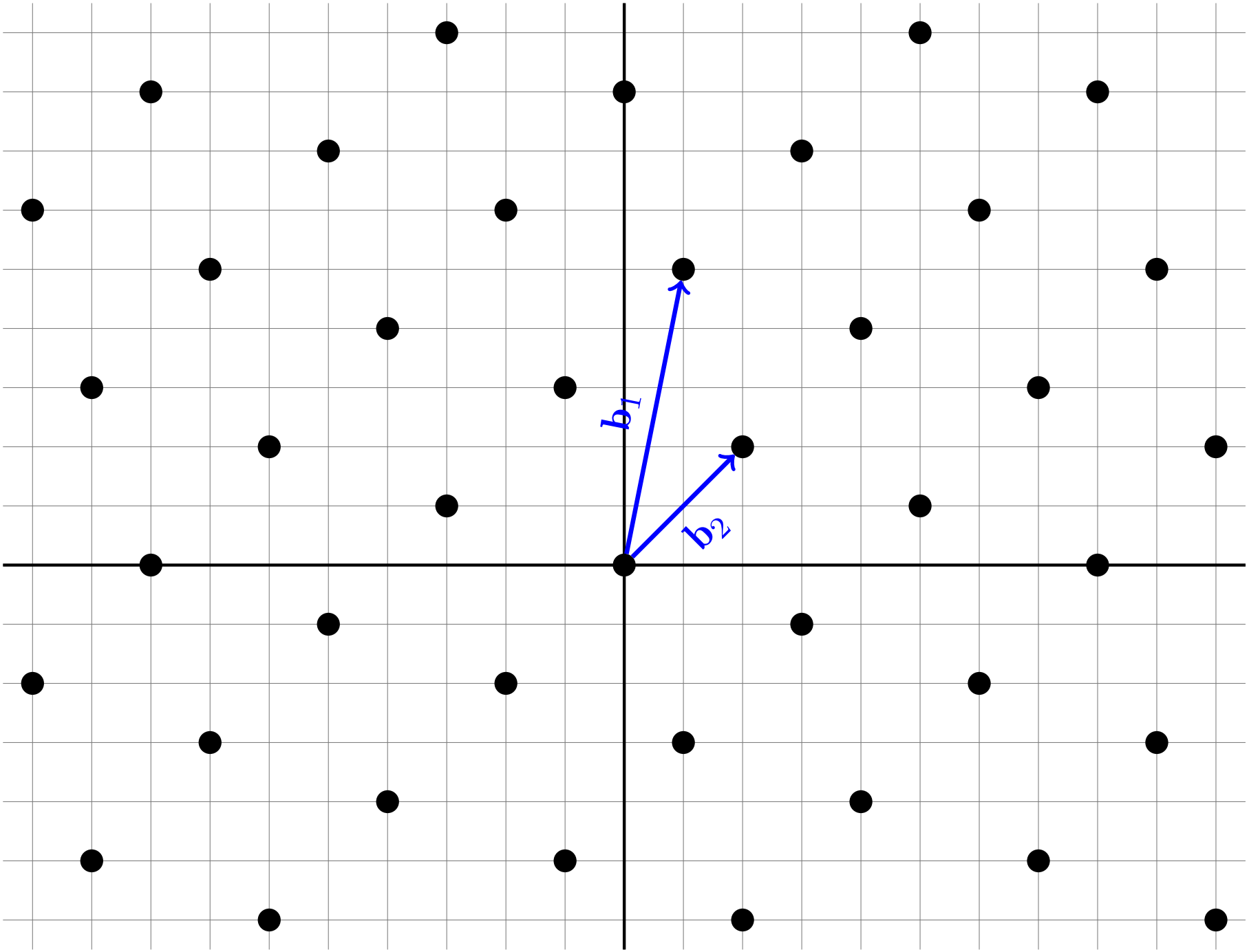
Definition (Réseau euclidien)

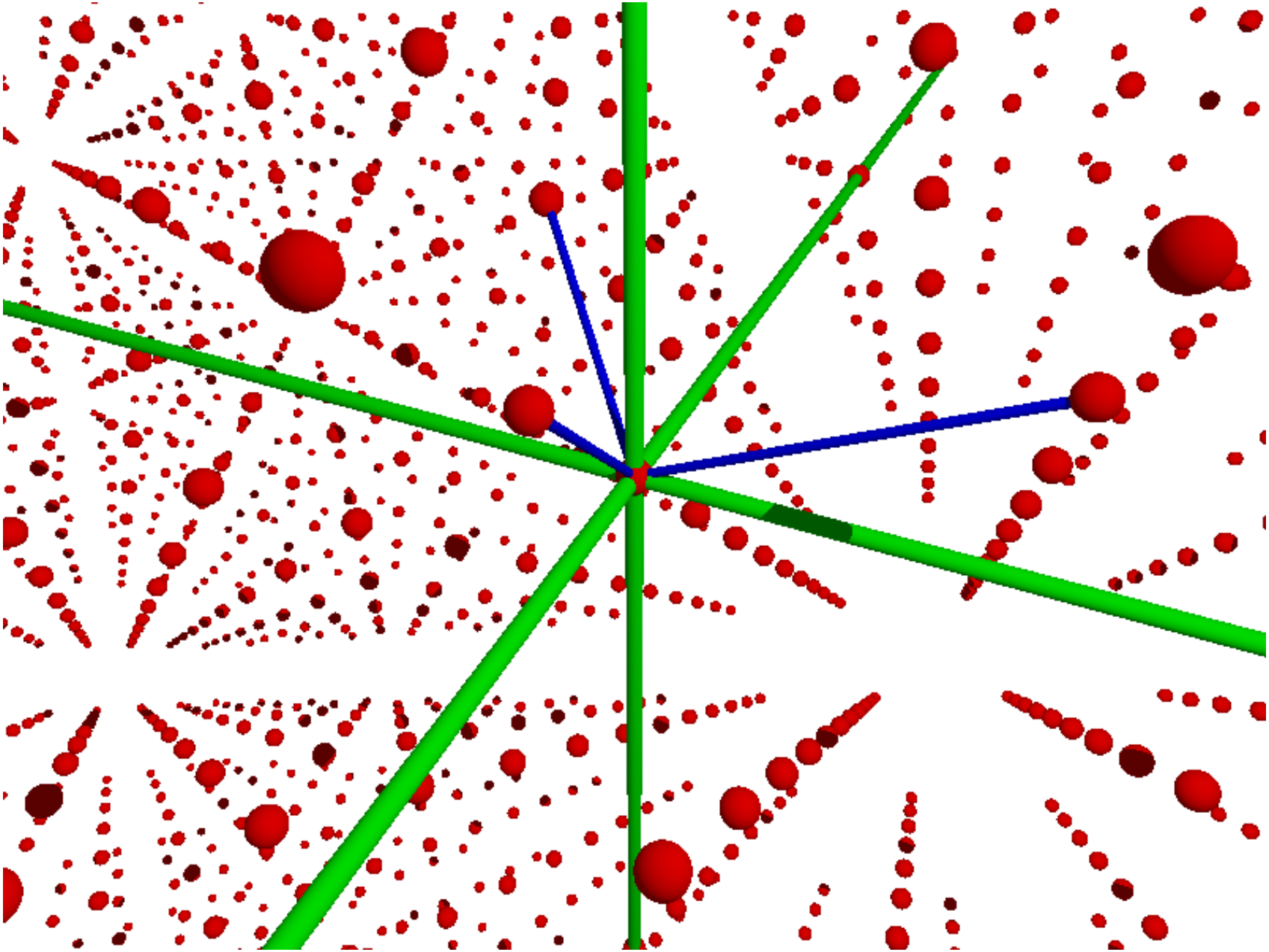
$\mathbf{b}_1, \ldots, \mathbf{b}_d$: vecteurs de $\mathbb{Z}^n$. Ils *engendrent* le réseau euclidien :

$$\mathcal{L} = \left\{ \sum_{i=1}^{d} \mu_i \mathbf{b}_i : (\mu_1, \ldots, \mu_d) \in \mathbb{Z}^d \right\}.$$

$\mathbf{b}_i$ linéairement indépendants $\rightsquigarrow$ *base* de $\mathcal{L}$ (d = dimension)
Si $d = n$, on dit que le réseau est *de rang plein*

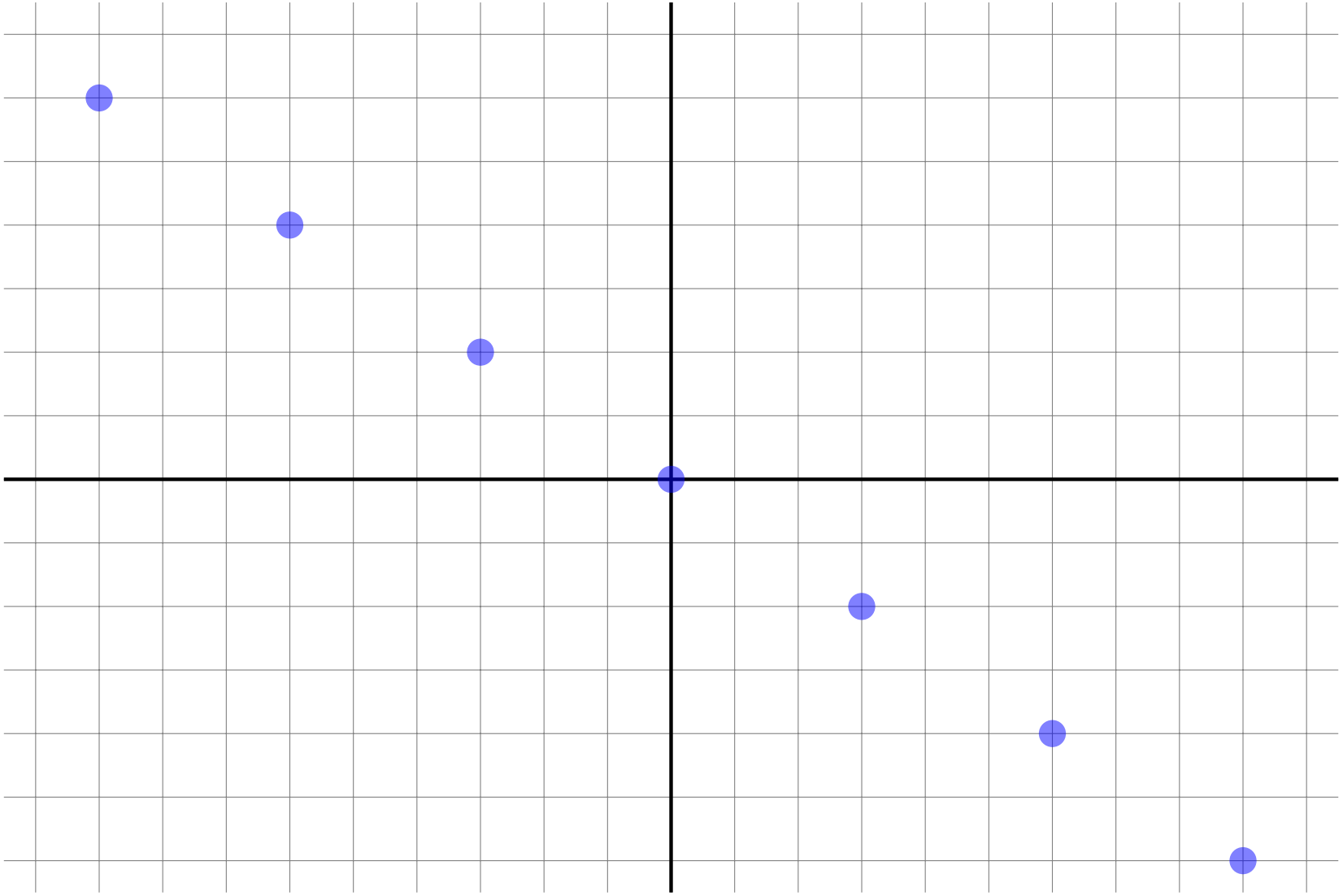
Ce sont les sous-groupes de $(\mathbb{Z}^n, +)$


$\mathbf{b}_1$
$\mathbf{b}_2$



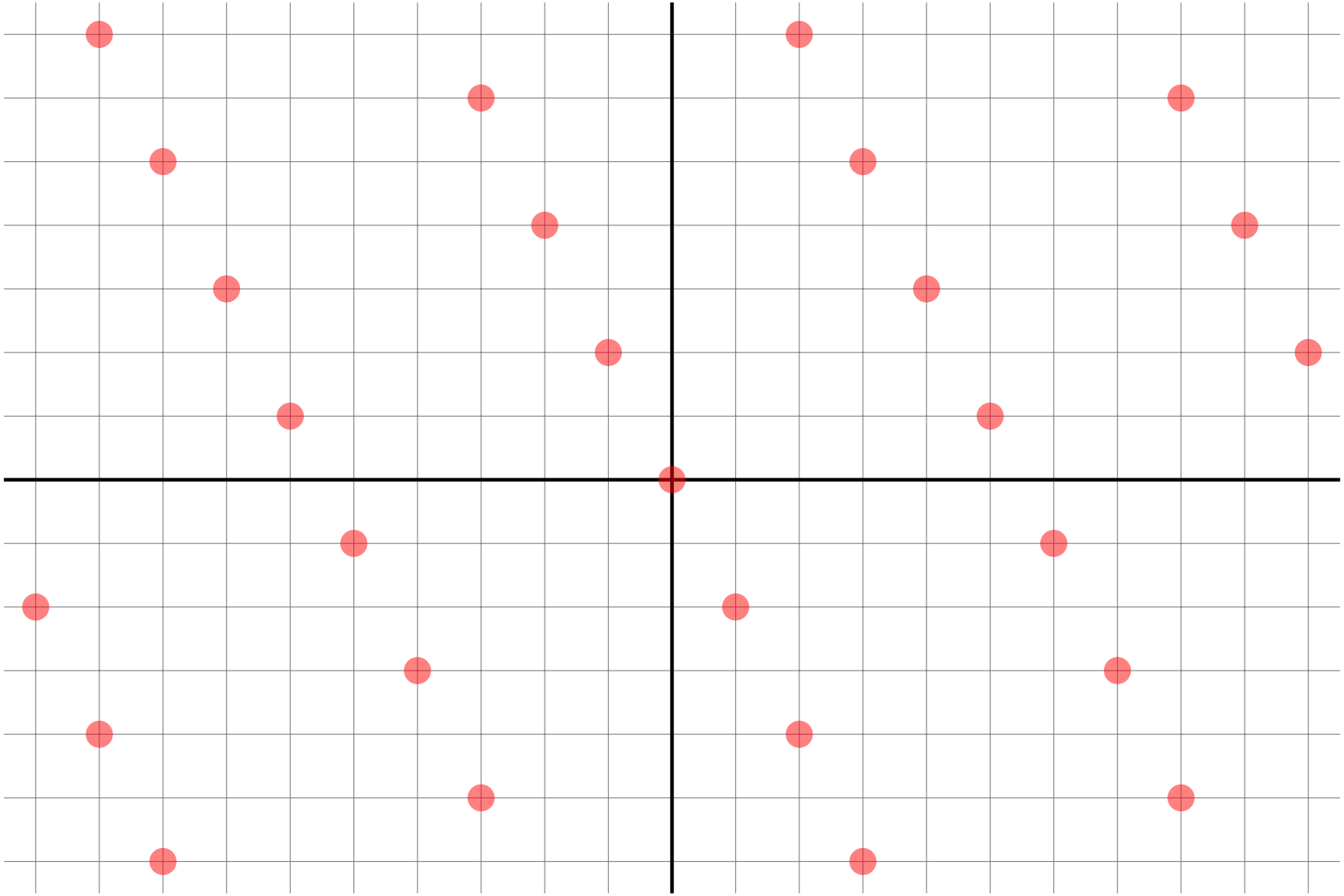
Équations diophantiennes linéaires

$$2x + 3y = 0$$



Équations diophantiennes linéaires

$$3x + 5y \equiv 0 \mod 11$$



Plan

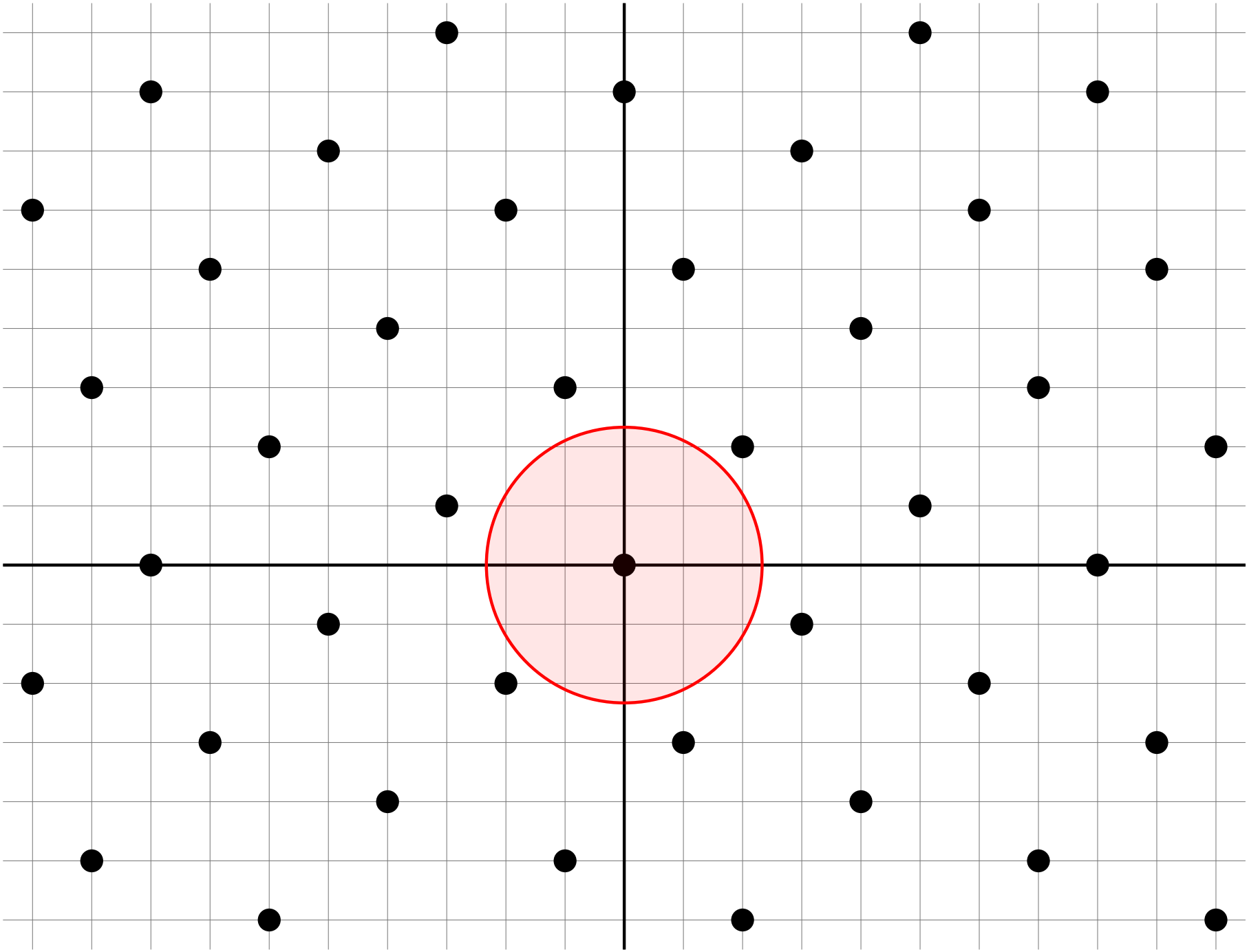
Teasing : sécurité de RSA avec Petits exposants secrets

Introduction aux réseaux euclidiens

Propriétés de base, plus court vecteur, volume

Problèmes algorithmiques dans les réseaux

Algorithmes de « réduction de réseau »
- En dimension 2 : réduction de Lagrange
- Algorithmes basés sur l'orthogonalisation
  - Orthogonalisation de Gram-Schmidt
  - La size-reduction

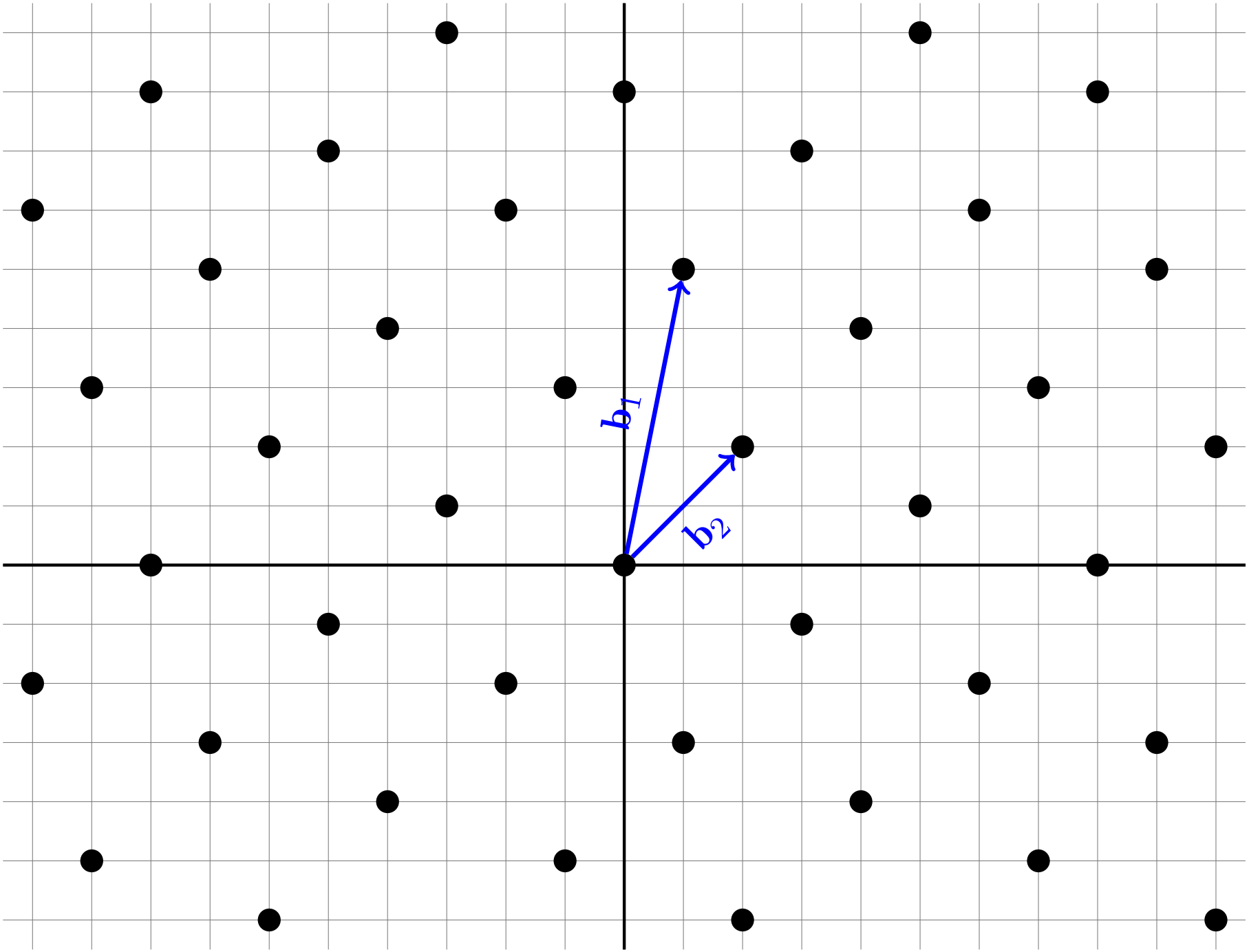


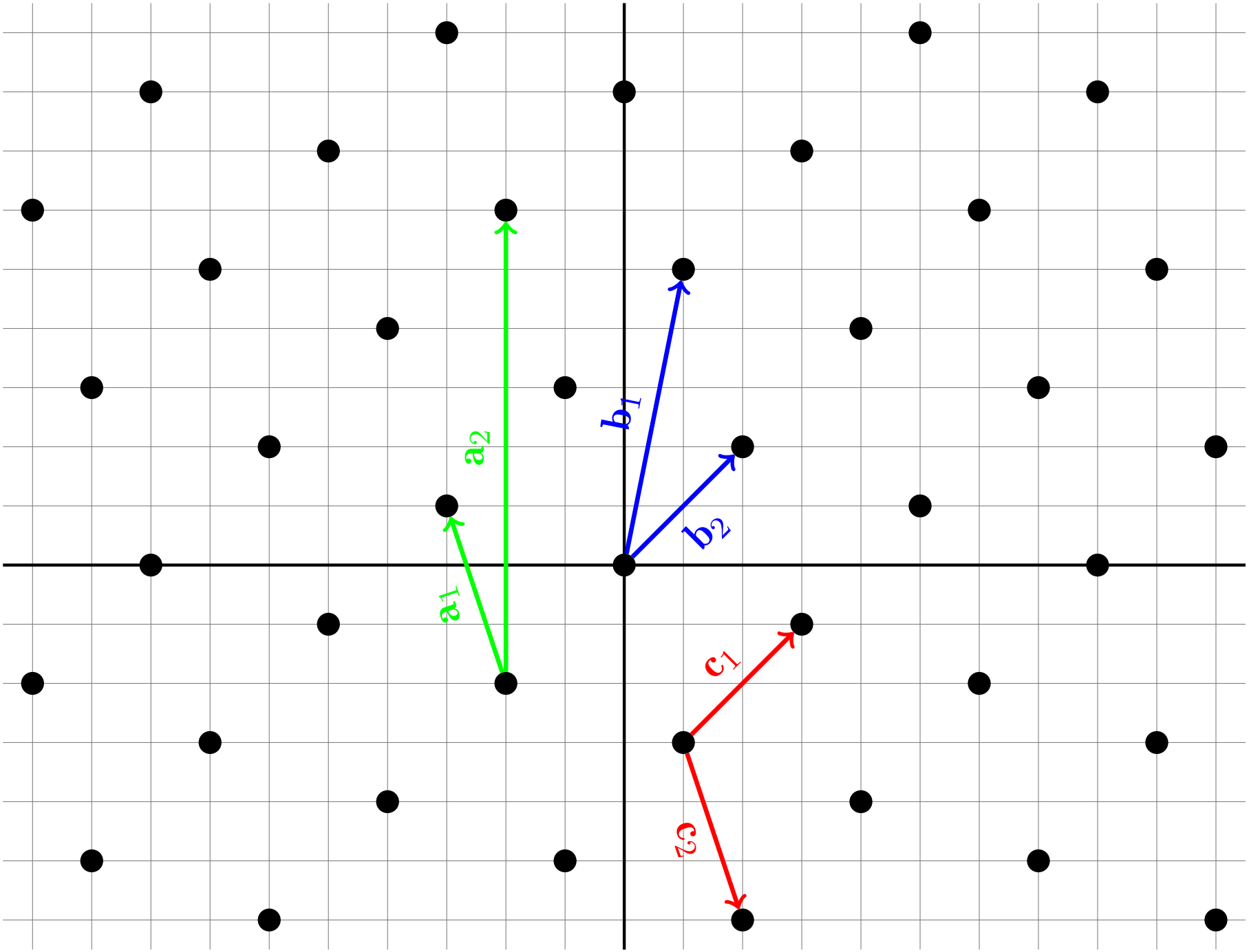
Plus court vecteur

- Un réseau possède un plus court vecteur (non-nul)
- Sa longueur : $\lambda_1(\mathcal{L})$ — caractéristique importante de $\mathcal{L}$

Stay Tuned

On peut apprendre des choses sur sa longueur sans trop d'effort


$\mathbf{b}_1$
$\mathbf{b}_2$


$\mathbf{a}_2$
$\mathbf{a}_1$
$\mathbf{b}_1$
$\mathbf{b}_2$
$\mathbf{c}_1$
$\mathbf{c}_2$

Bases

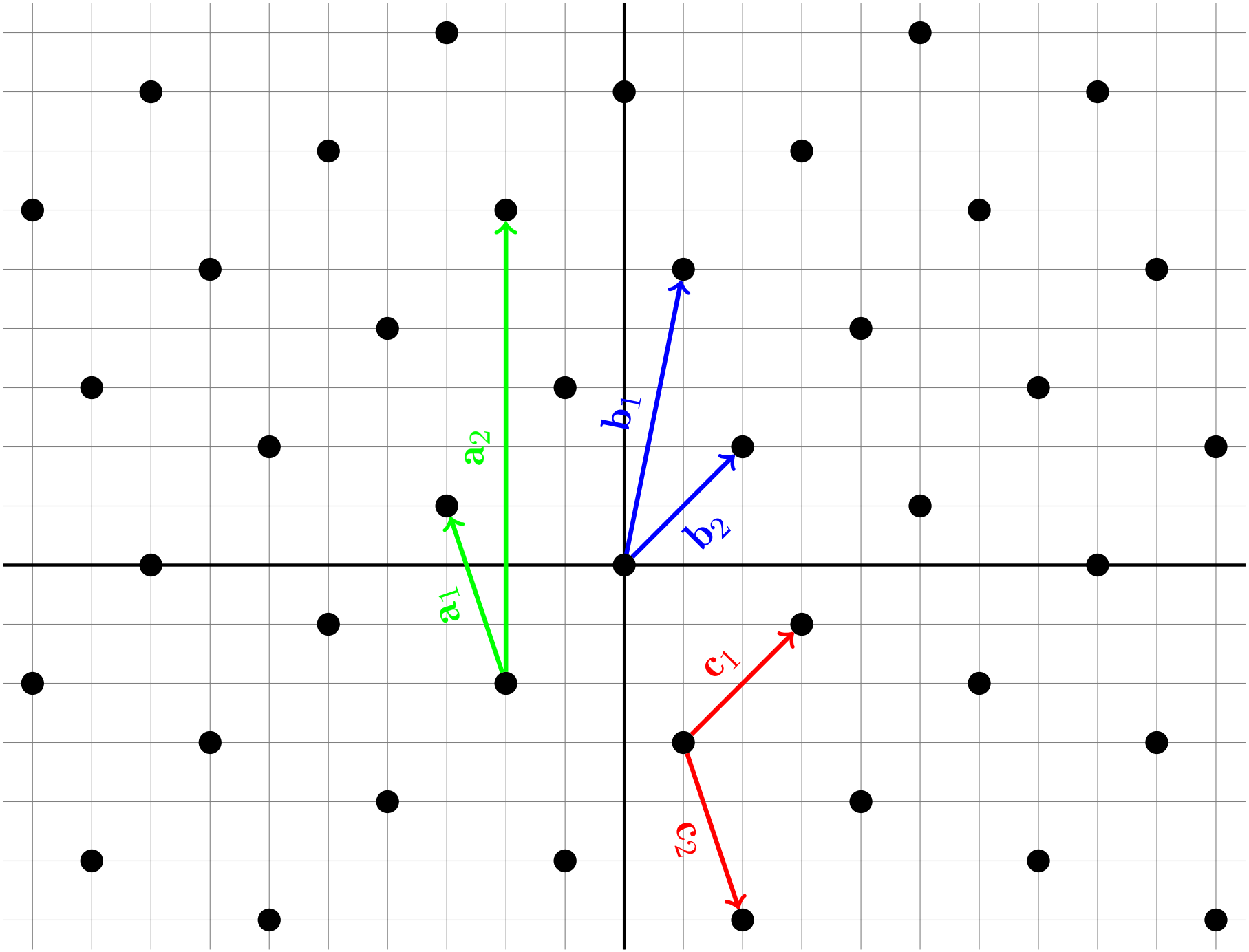
- Réseau précédent engendré par $\mathbf{b_1} = (1, 5), \mathbf{b_2} = (2, 2)$
- Représenté de manière commode par :

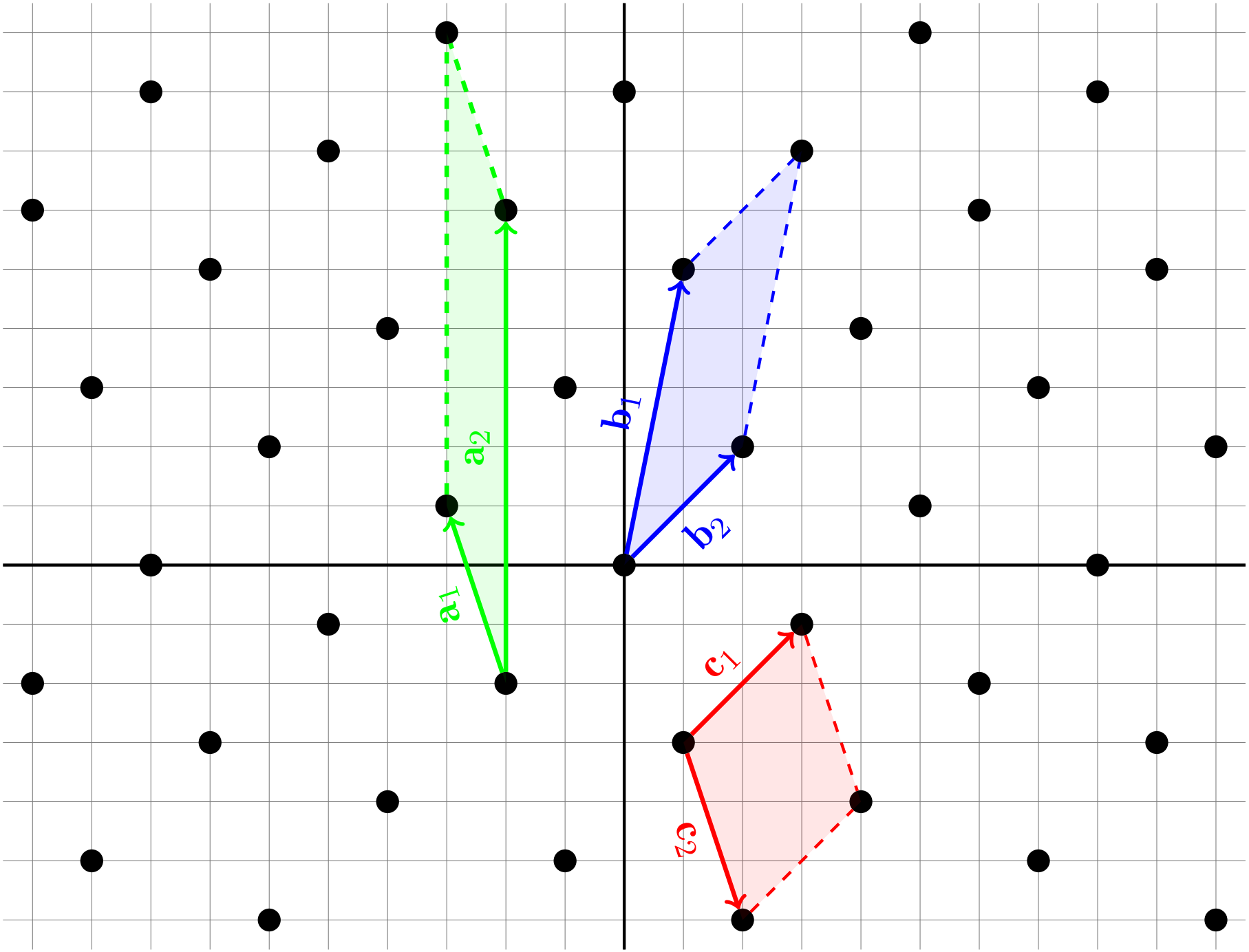
$$B = \begin{pmatrix} 1 & 5 \\ 2 & 2 \end{pmatrix}$$

  Alors $\mathcal{L} = \{\mathbf{x} \cdot B \text{ avec } \mathbf{x} \in \mathbb{Z}^d\}$.
- Plusieurs bases possibles. Par ex. :

$$A = \begin{pmatrix} -1 & 3 \\ 0 & 8 \end{pmatrix} \quad \text{et} \quad C = \begin{pmatrix} 2 & 2 \\ 1 & -3 \end{pmatrix}$$

- A, B deux bases de $\mathcal{L} \iff A = U \times B$ avec $\det U = \pm 1$


$\mathbf{a}_2$
$\mathbf{b}_1$
$\mathbf{b}_2$
$\mathbf{a}_1$
$\mathbf{c}_1$
$\mathbf{c}_2$


$\mathbf{a}_1$
$\mathbf{a}_2$
$\mathbf{b}_1$
$\mathbf{b}_2$
$\mathbf{c}_1$
$\mathbf{c}_2$

On fixe une base $(\mathbf{b}_1, \ldots, \mathbf{b}_d)$

- parallélepipède fondamental :

$$\left\{ \sum_{i=1}^{d} x_i \mathbf{b}_i : x_1, \ldots, x_d \in [0; 1[\right\}$$

- Son **volume** ne dépend pas du choix de la base

$\Rightarrow$ caractéristique (importante) du réseau

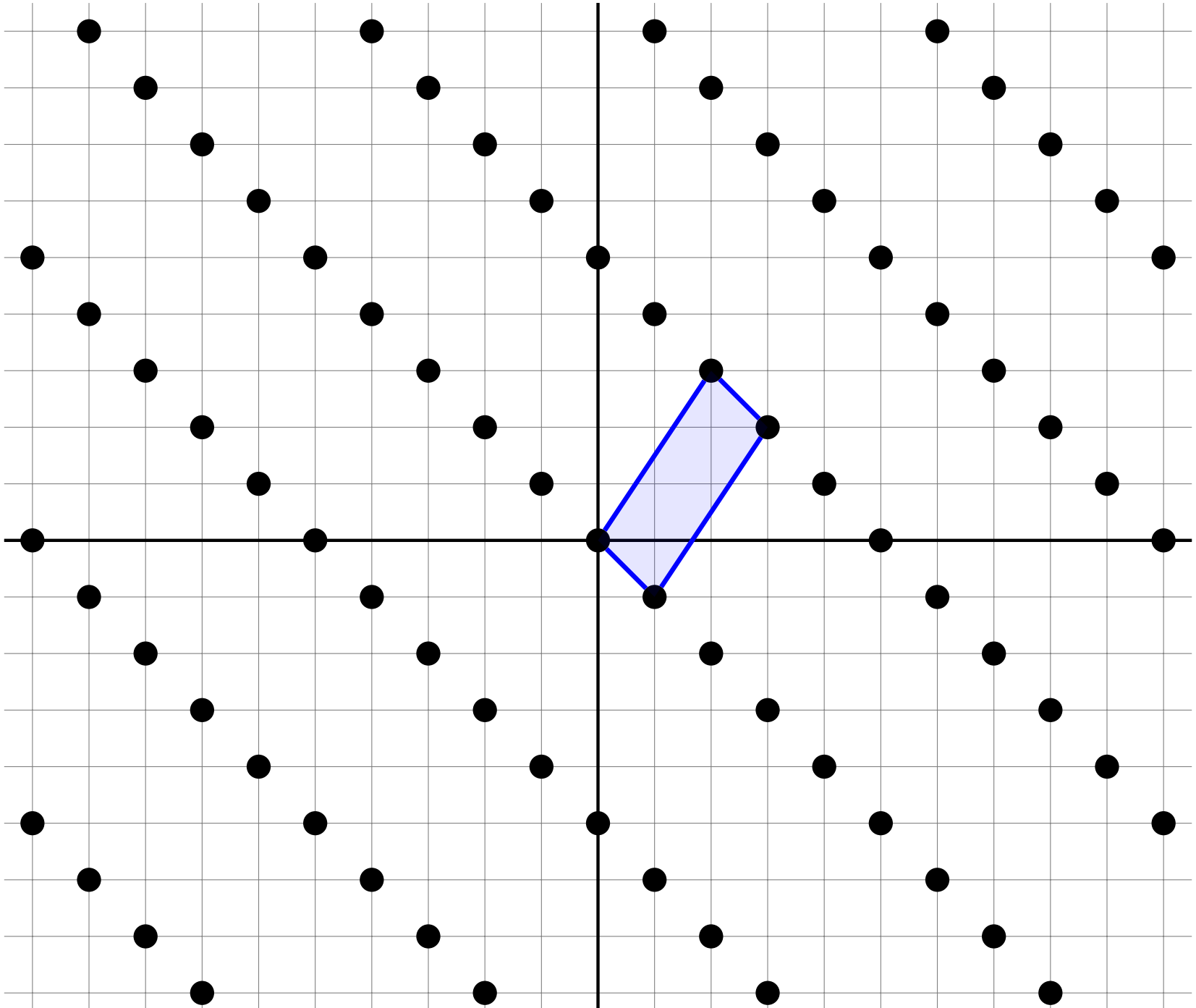
Theorem

Soit $\mathcal{L}$ un réseau de base B. On a $\mathrm{Vol}(\mathcal{L}) = \sqrt{\det BB^t}$

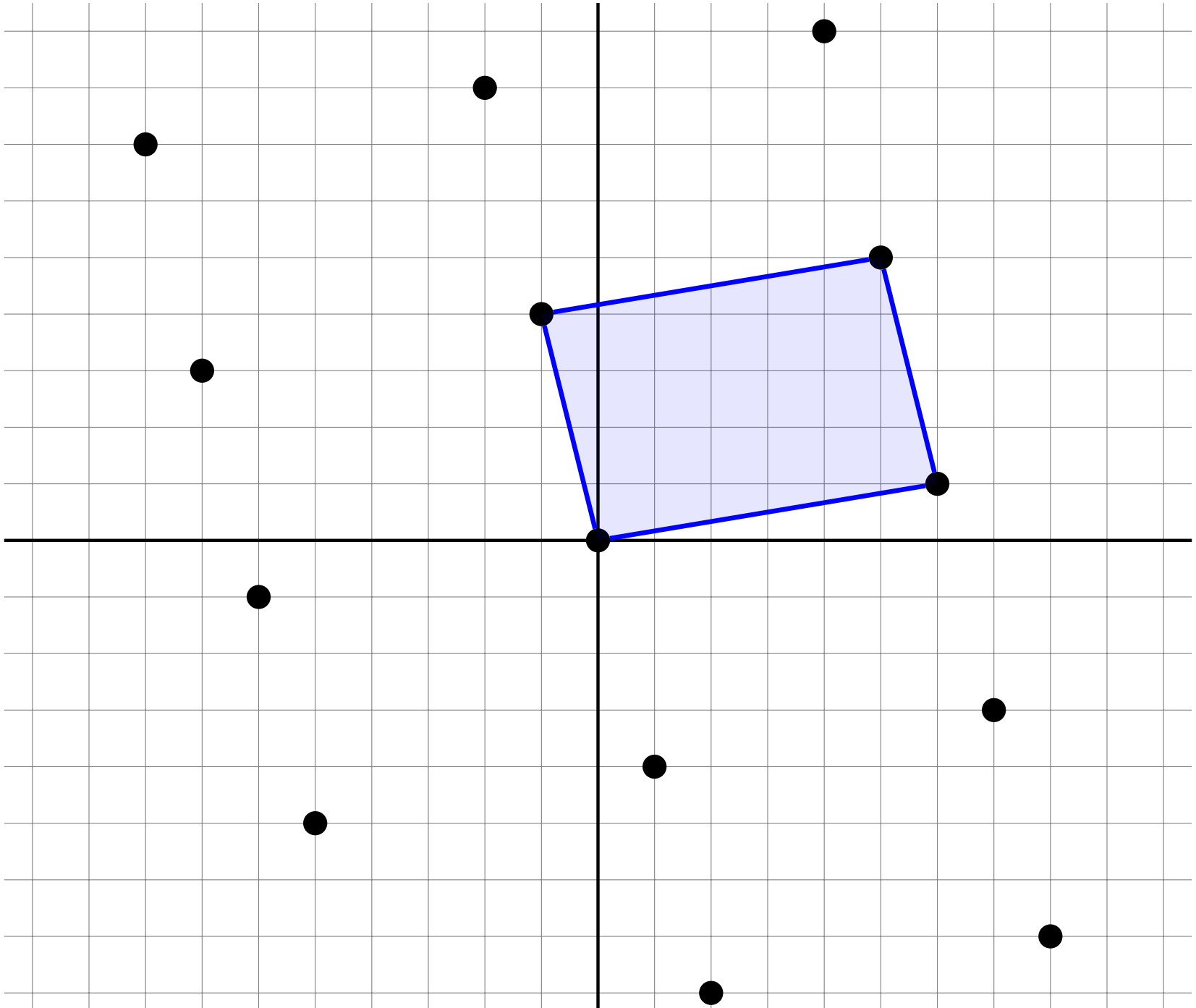
Si $\mathcal{L}$ est de rang plein, ça se simplifie en $\mathrm{Vol}(\mathcal{L}) = |\det B|$

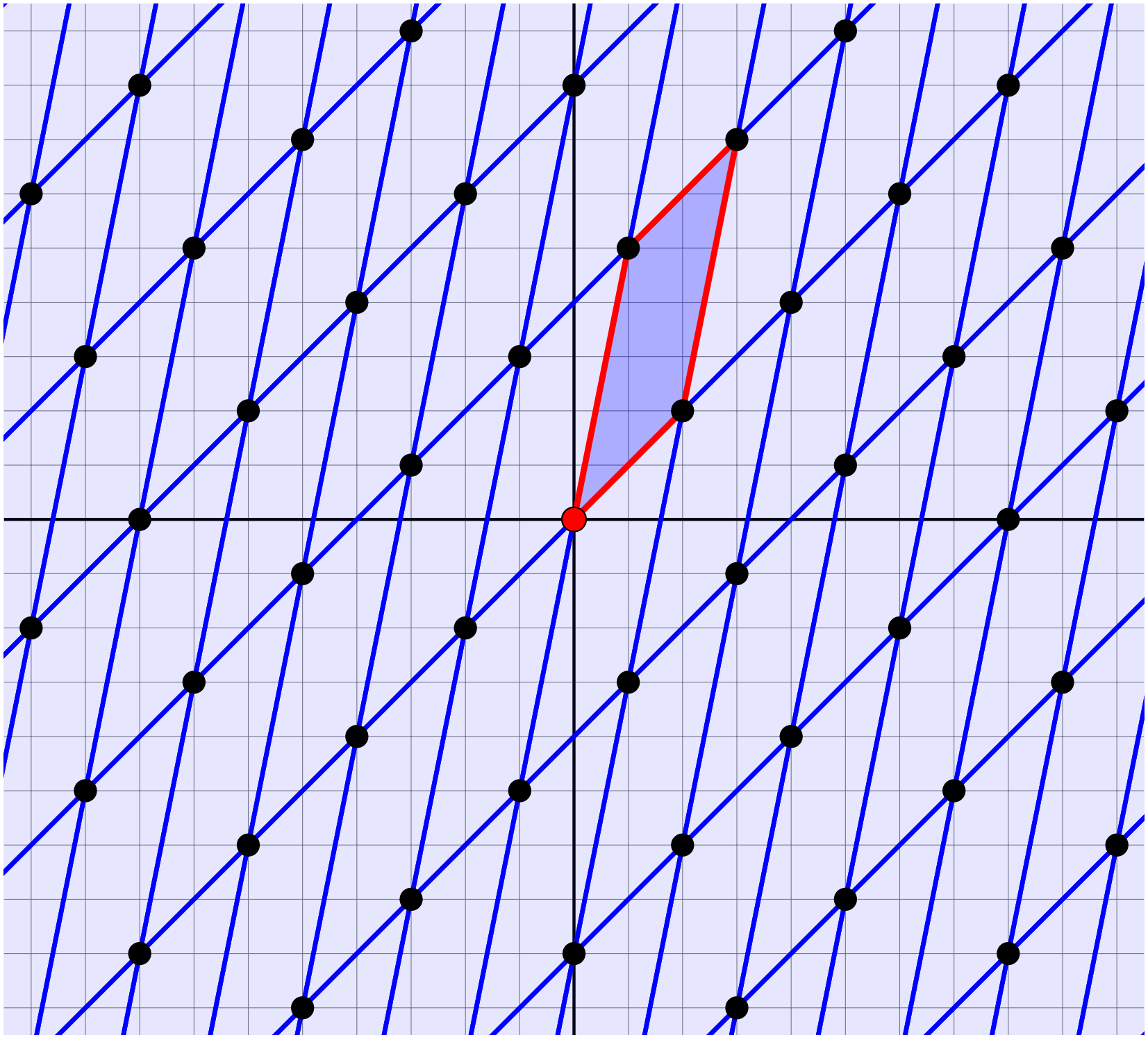
$\Rightarrow$ Le volume est (souvent) facile à déterminer

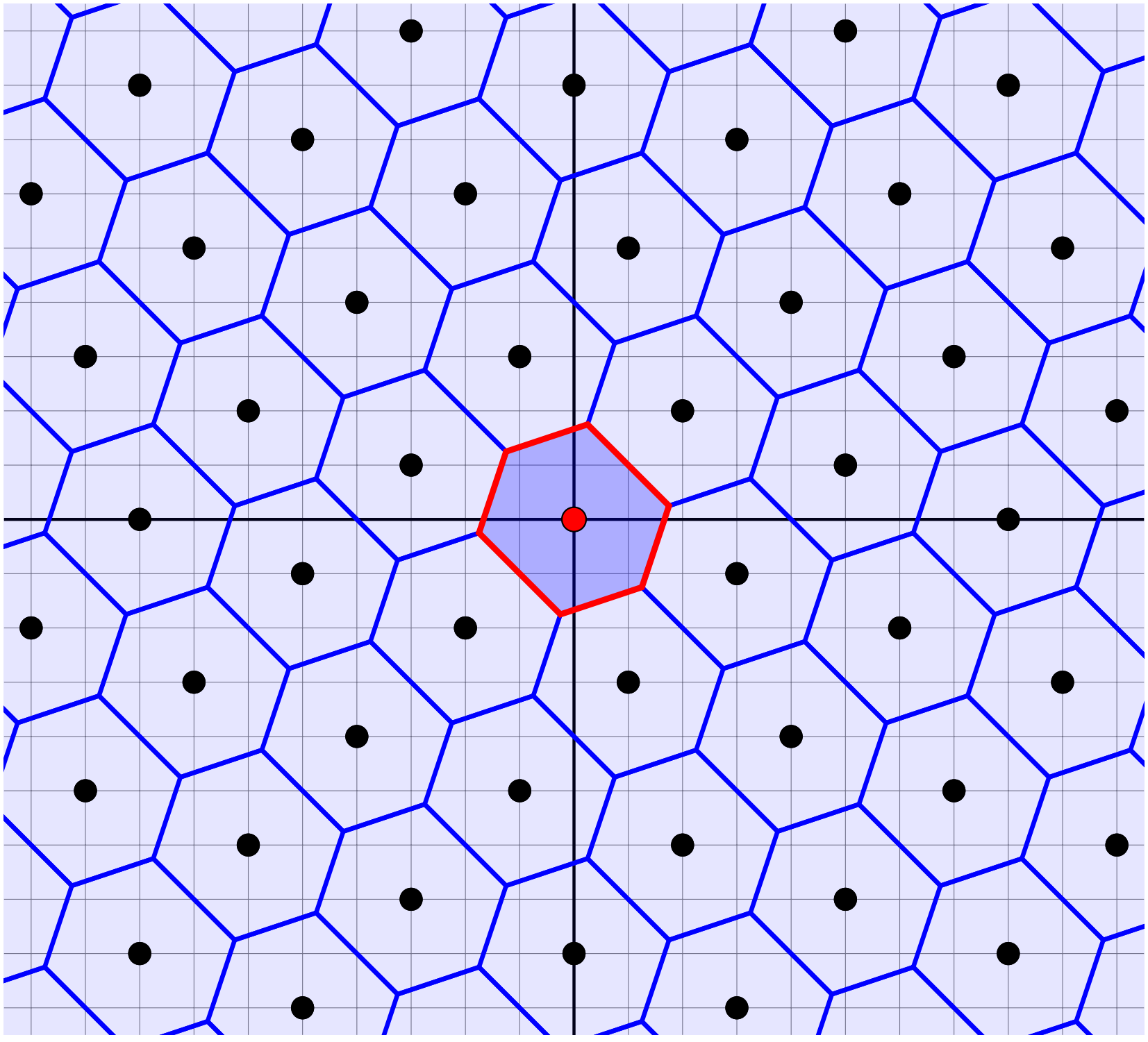
Volume = 5

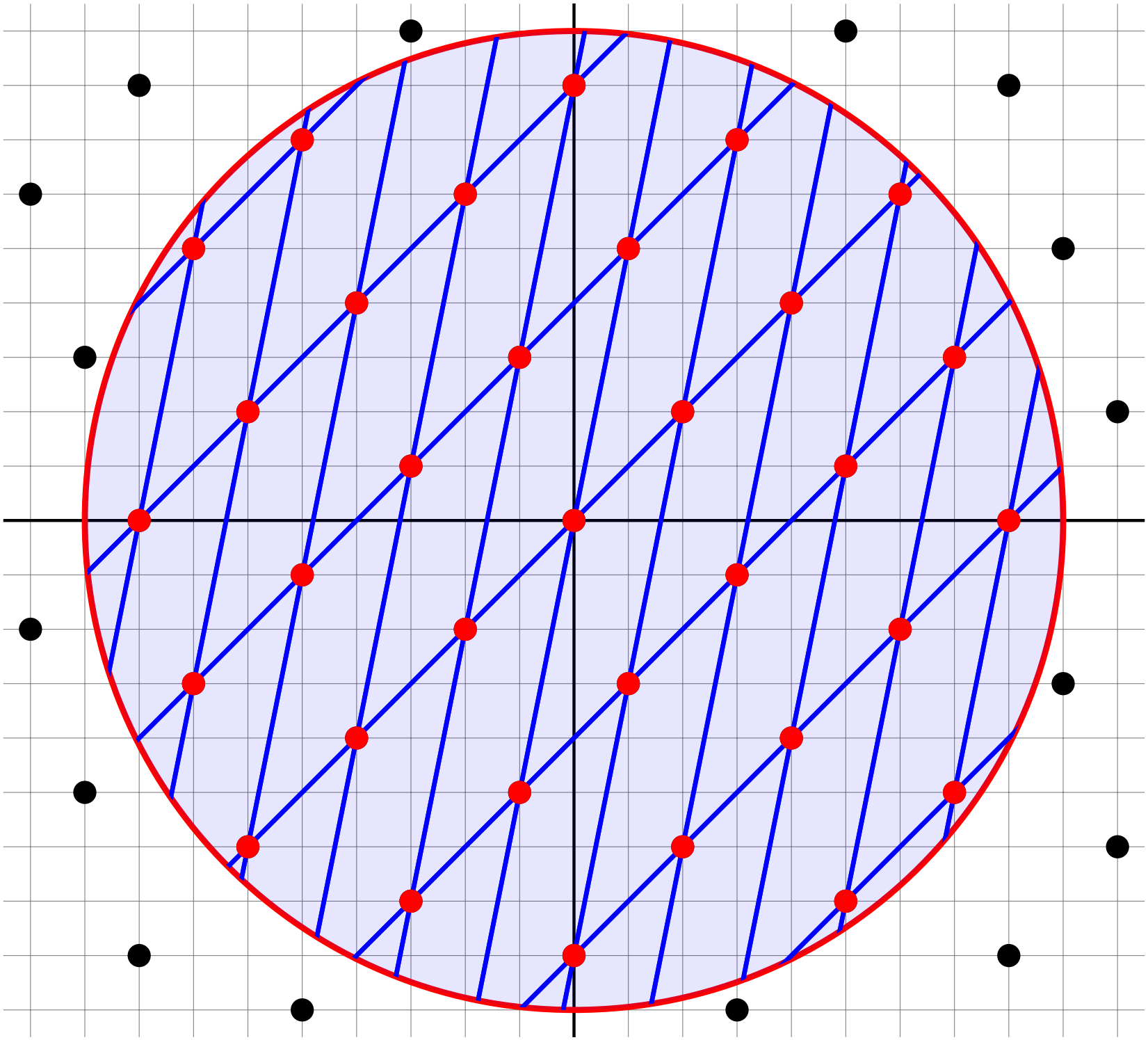


Volume = 31









Heuristique gaussienne

Un des principaux intérêts du concept de volume

$$\#\text{ points dans la boule} \approx \frac{\text{Volume de la boule}}{\text{Volume du réseau}}$$

« Boule » = ensembles des points à distance $\leq r$ de l'origine

Plus court vecteur

- Un réseau possède un plus court vecteur (non-nul)
- Sa longueur : $\lambda_1(\mathcal{L})$ — caractéristique importante de $\mathcal{L}$

Heuristique gaussienne

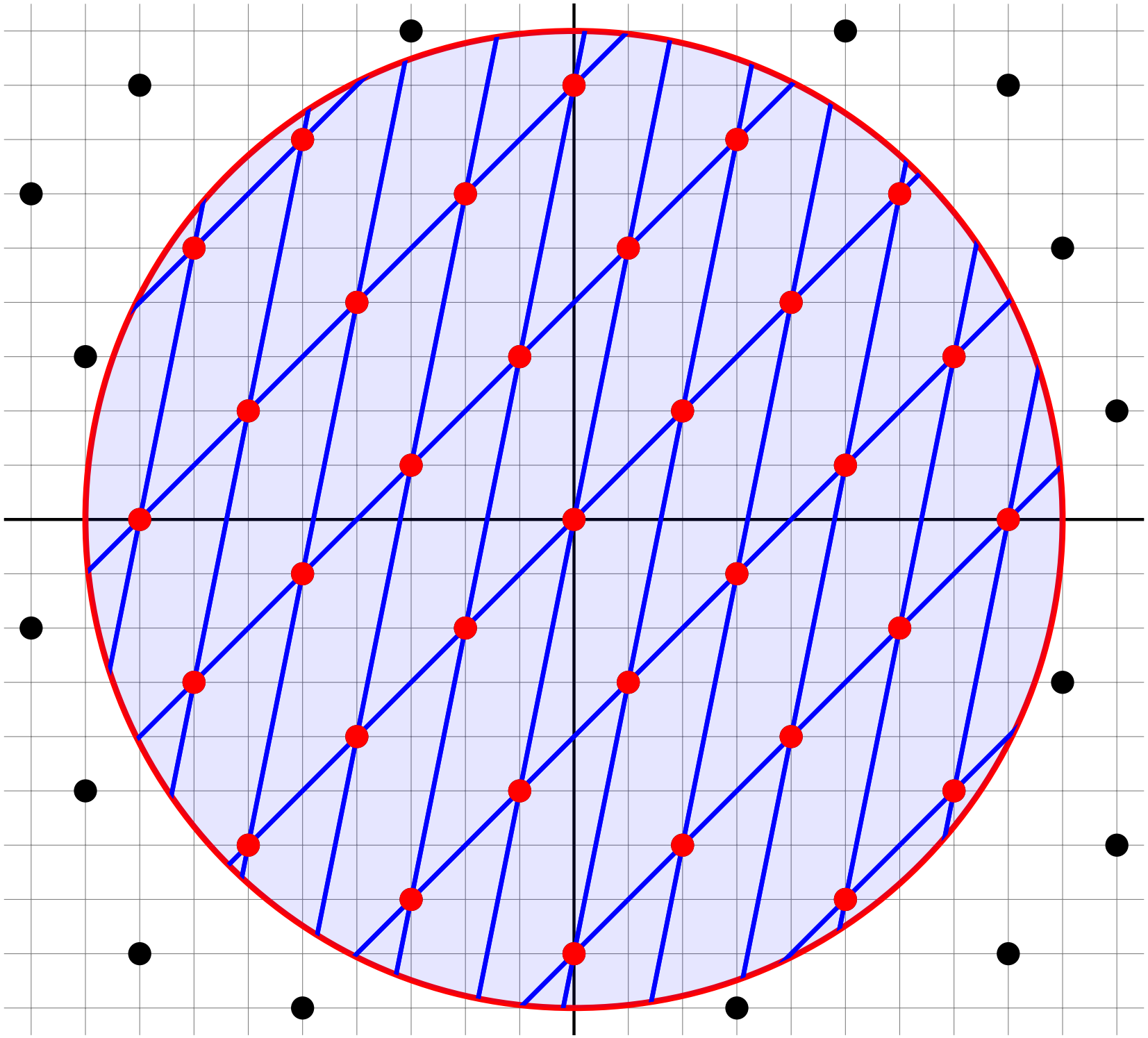
$$\#\text{ points dans la boule} \approx \frac{\text{Volume de la boule}}{\text{Volume du réseau}}$$

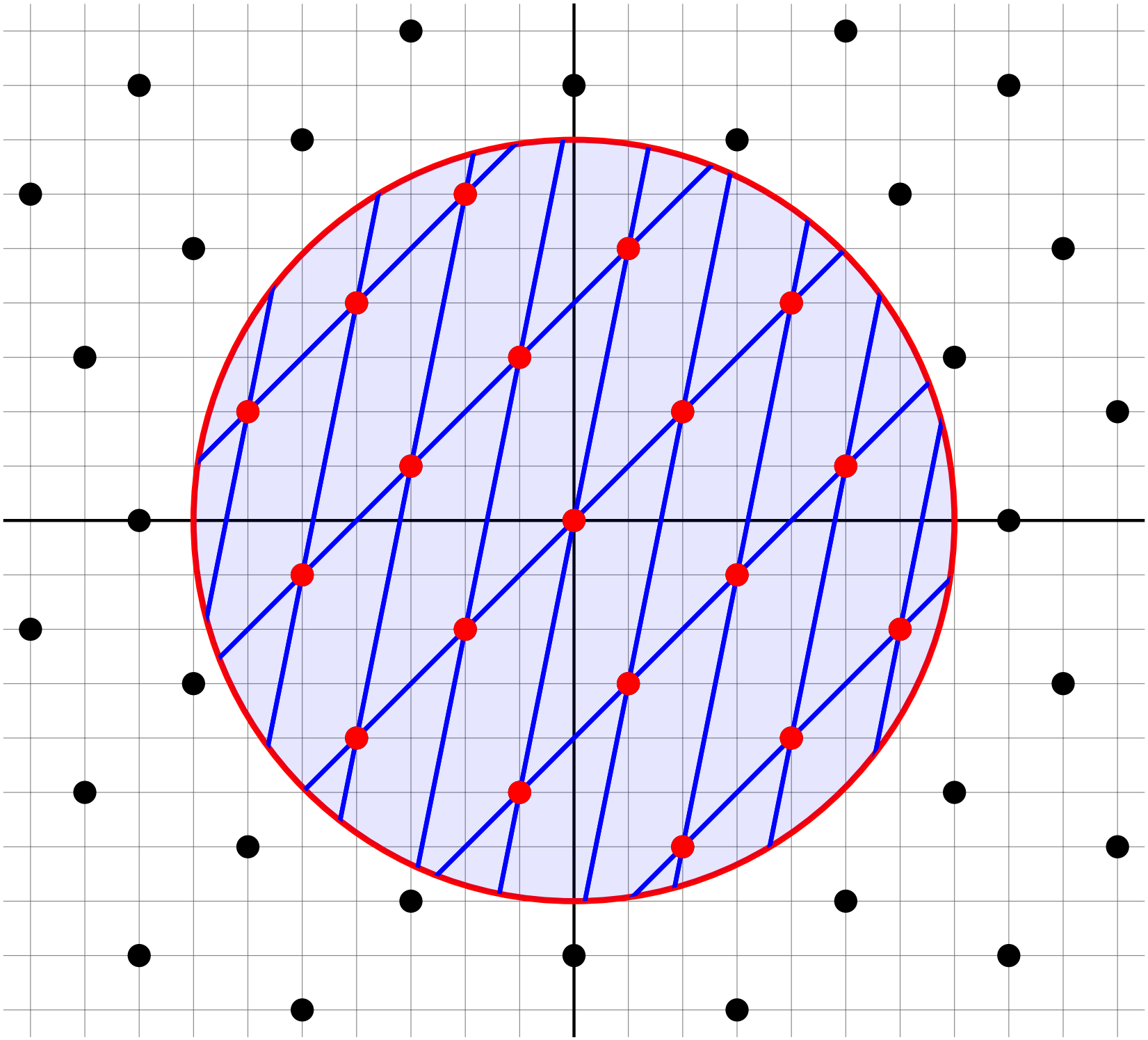
Le + grand rayon pour lequel la boule de rayon r ne contient plus qu'un point est (à peu près) la taille du plus court vecteur

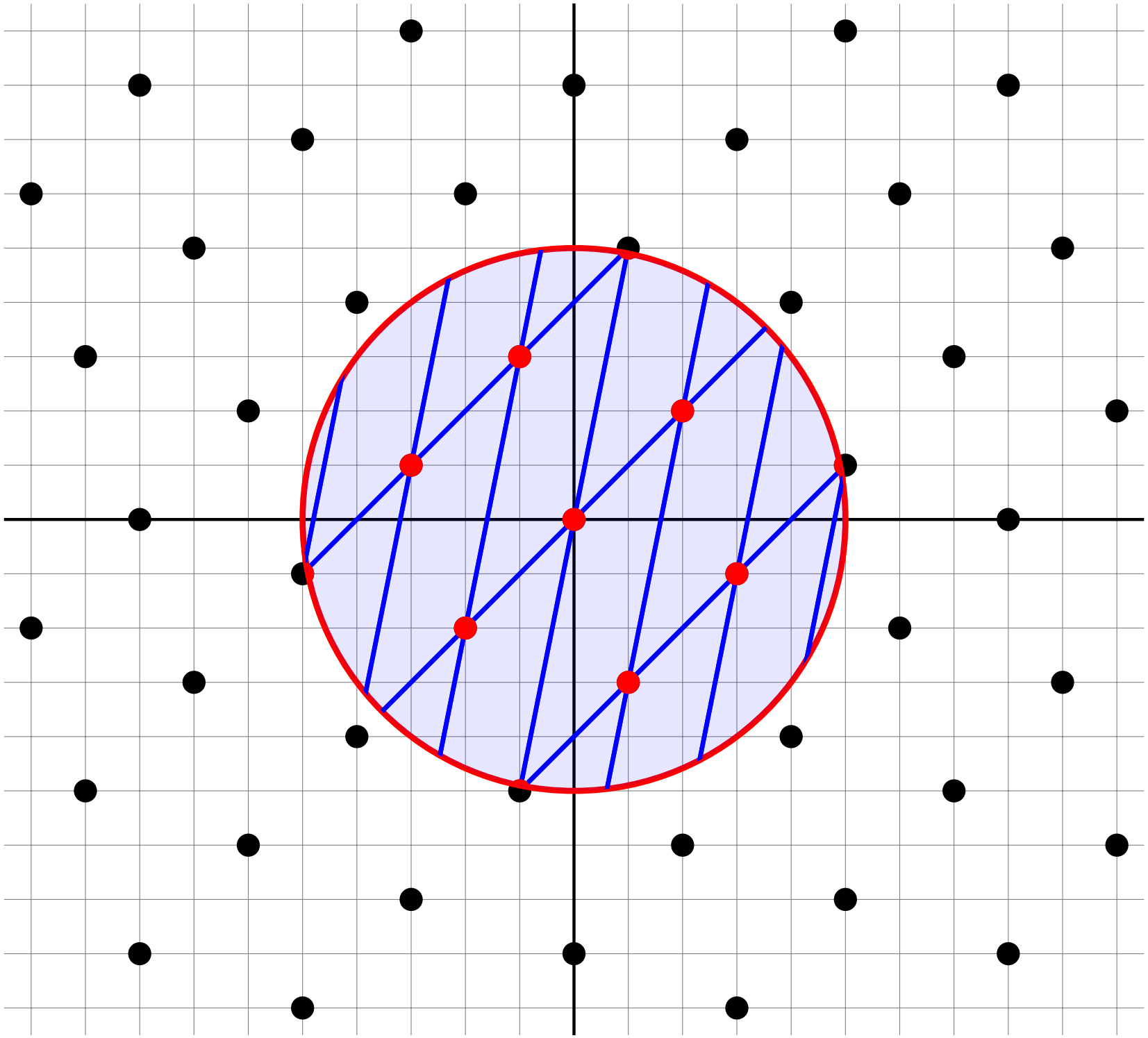
$$\lambda_1(\mathcal{L}) \approx \sqrt{n}\,(\mathrm{Vol}(\mathcal{L}))^{\frac{1}{n}}$$

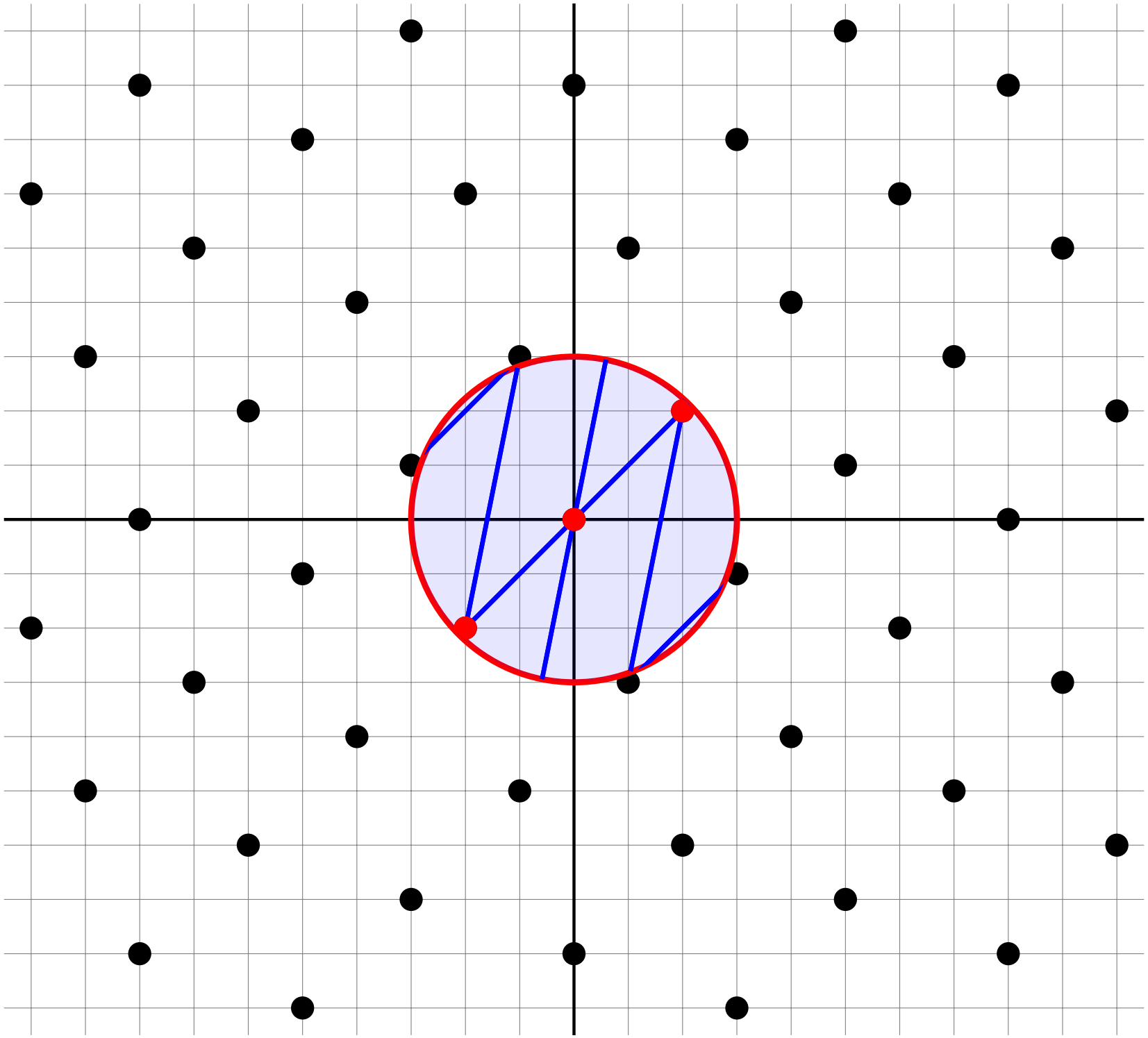
Theorem (Borne de Minkowski simplifiée)

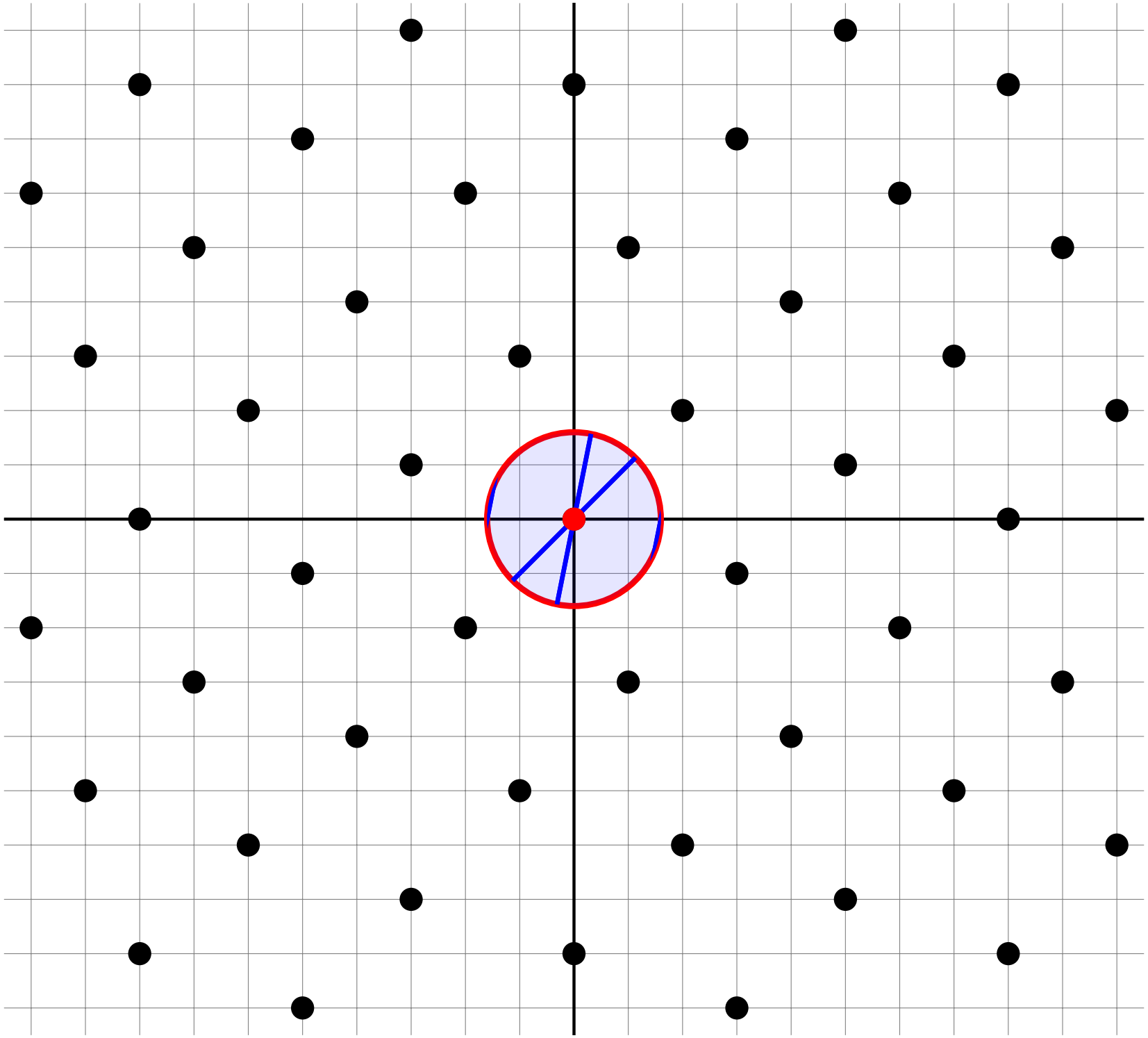
Si $\mathcal{L}$ est un réseau de dimension d, alors $\lambda_1(\mathcal{L}) \leq \sqrt{d}\,(\mathrm{Vol}(\mathcal{L}))^{\frac{1}{d}}$

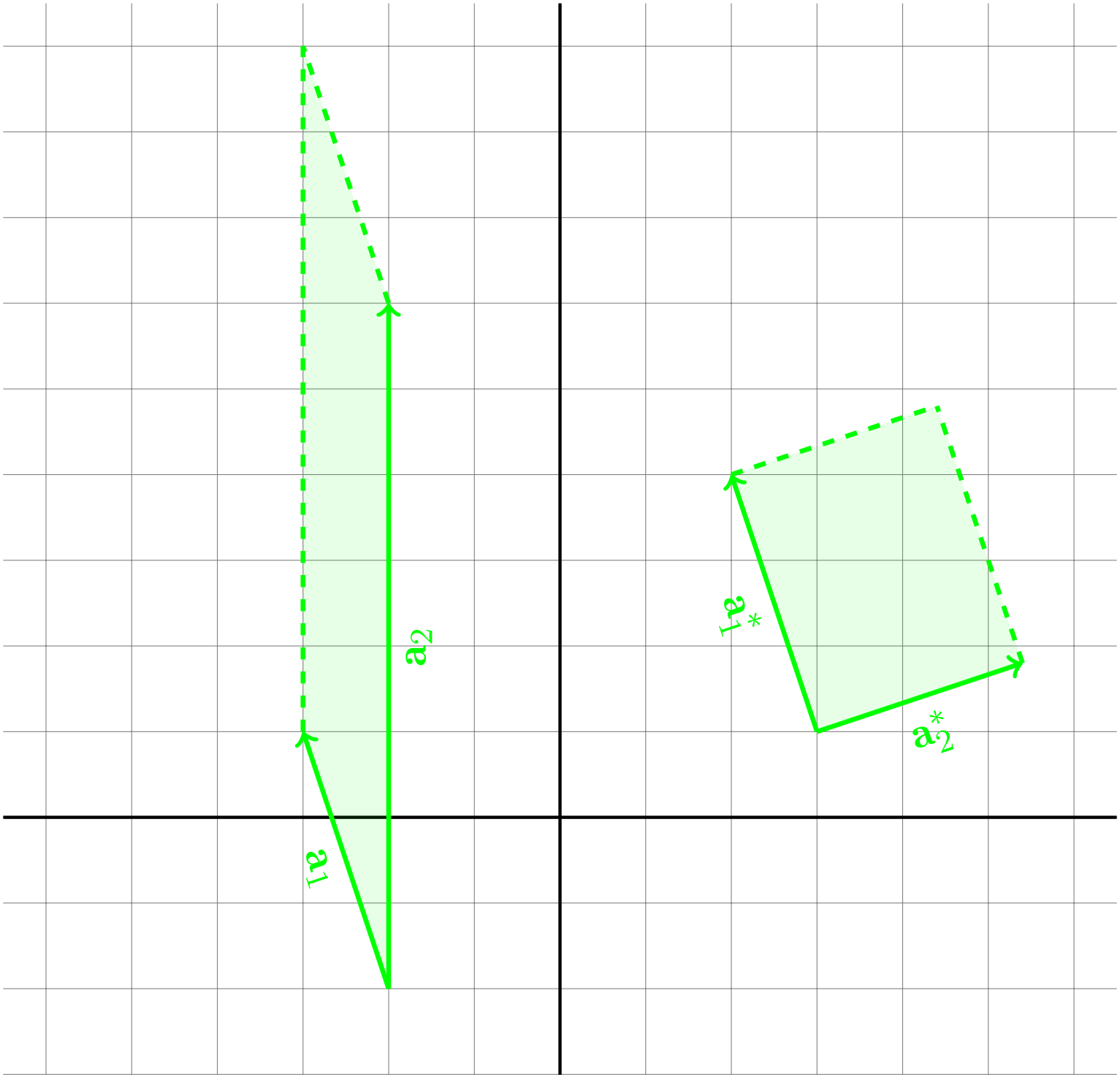











$\mathbf{a}_2$
$\mathbf{a}_1$
$\mathbf{a}_1^*$
$\mathbf{a}_2^*$

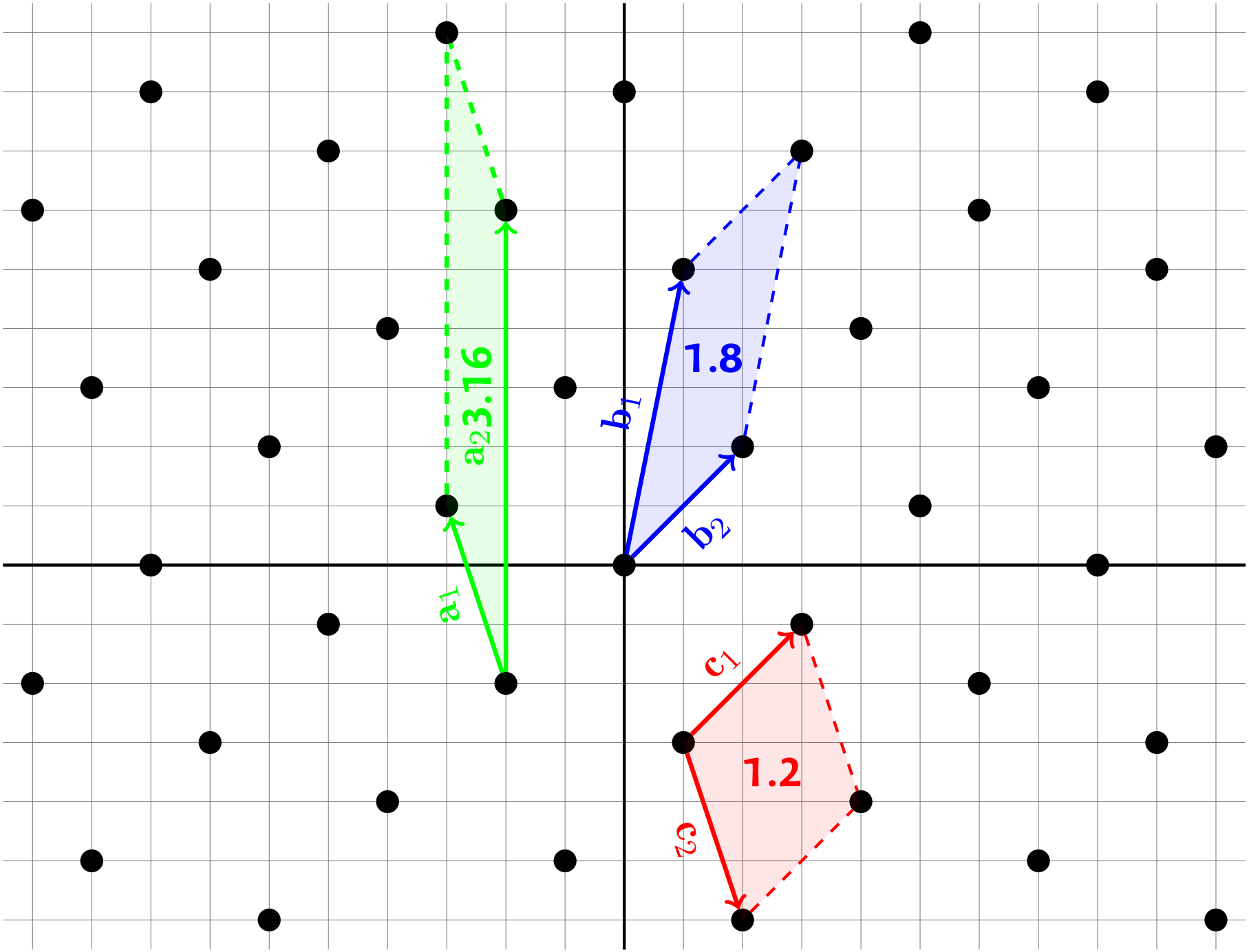
Orthogonalité

- Orthogonal = « perpendiculaire »
- Deux vecteurs $\mathbf{x}$ et $\mathbf{y}$ sont orthogonaux ssi $\langle \mathbf{x}, \mathbf{y} \rangle = 0$
- Un réseau n'admet pas forcément (et même rarement) une base orthogonale

Défaut d'orthogonalité

$$\frac{\|\mathbf{b}_1\| \times \cdots \times \|\mathbf{b}_d\|}{\mathrm{Vol}(\mathcal{L})}$$

- Plus il est grand, moins la base est orthogonale / courte
- Il vaut 1 sur un parallélépipède


a2
3.16
a1
b1
1.8
b2
c1
1.2
c2

Plan

Teasing : sécurité de RSA avec Petits exposants secrets

Introduction aux réseaux euclidiens

Propriétés de base, plus court vecteur, volume

Problèmes algorithmiques dans les réseaux

Algorithmes de « réduction de réseau »
- En dimension 2 : réduction de Lagrange
- Algorithmes basés sur l'orthogonalisation
  - Orthogonalisation de Gram-Schmidt
  - La size-reduction

Les classiques

Shortest Vector Problem (SVP) — **NP-dur**

Étant donné une base d'un réseau $\mathcal{L}$, calculer un vecteur non-nul $\mathbf{x}$ tel que $\|\mathbf{x}\| = \lambda_1(\mathcal{L})$.

Closest Vector Problem (CVP) — **NP-dur**

Étant donné une base d'un réseau $\mathcal{L}$ et un vecteur $\mathbf{y}$, calculer un vecteur $\mathbf{x}$ tel que $\|\mathbf{y} - \mathbf{x}\| \leq \|\mathbf{y} - \mathbf{z}\|$ pour tout $\mathbf{z} \in \mathcal{L}$.

Les classiques

Approximate Shortest Vector Problem (SVP_γ)

Étant donné une base d'un réseau $\mathcal{L}$, calculer un vecteur non-nul $\mathbf{x}$ tel que $\|\mathbf{x}\| \leq \gamma \times \lambda_1(\mathcal{L})$.

Approximate Closest Vector Problem (CVP_γ)

Étant donné une base d'un réseau $\mathcal{L}$ et un vecteur $\mathbf{y}$, calculer un vecteur $\mathbf{x}$ tel que $\|\mathbf{y} - \mathbf{x}\| \leq \gamma \times \|\mathbf{y} - \mathbf{z}\|$ pour tout $\mathbf{z} \in \mathcal{L}$.

Les classiques

Approximate Shortest Vector Problem (SVP_γ)

Étant donné une base d'un réseau $\mathcal{L}$, calculer un vecteur non-nul $\mathbf{x}$ tel que $\|\mathbf{x}\| \leq \gamma \times \lambda_1(\mathcal{L})$.

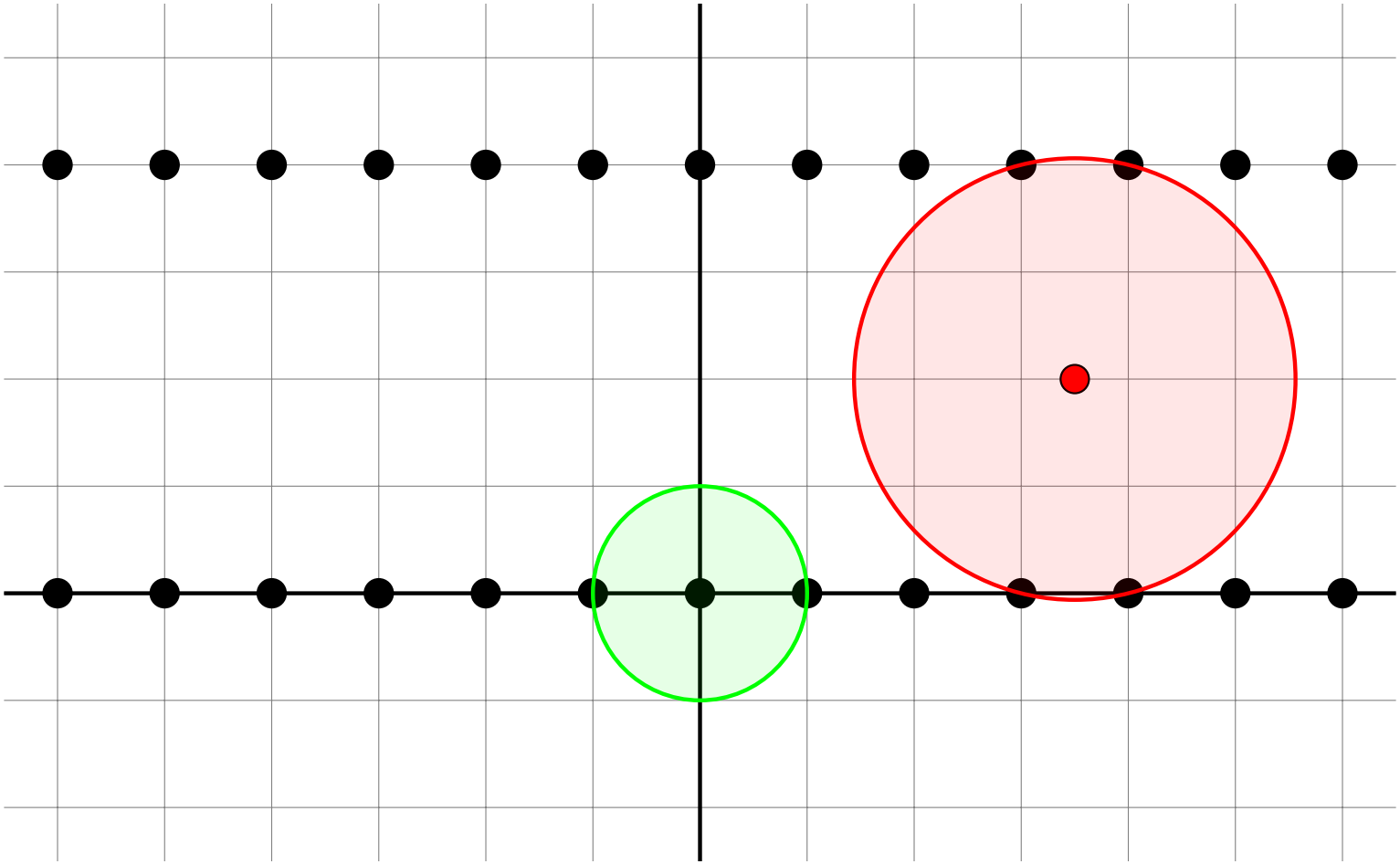
Hermite Shortest Vector Problem (HSVP_γ)

Étant donné une base d'un réseau $\mathcal{L}$, calculer un vecteur non-nul $\mathbf{x}$ tel que $\|\mathbf{x}\| \leq \gamma \times \mathrm{Vol}(\mathcal{L})^{1/n}$.

Approximate Closest Vector Problem (CVP_γ)

Étant donné une base d'un réseau $\mathcal{L}$ et un vecteur $\mathbf{y}$, calculer un vecteur $\mathbf{x}$ tel que $\|\mathbf{y} - \mathbf{x}\| \leq \gamma \times \|\mathbf{y} - \mathbf{z}\|$ pour tout $\mathbf{z} \in \mathcal{L}$.

SVP vs CVP



CVP est NP-dur

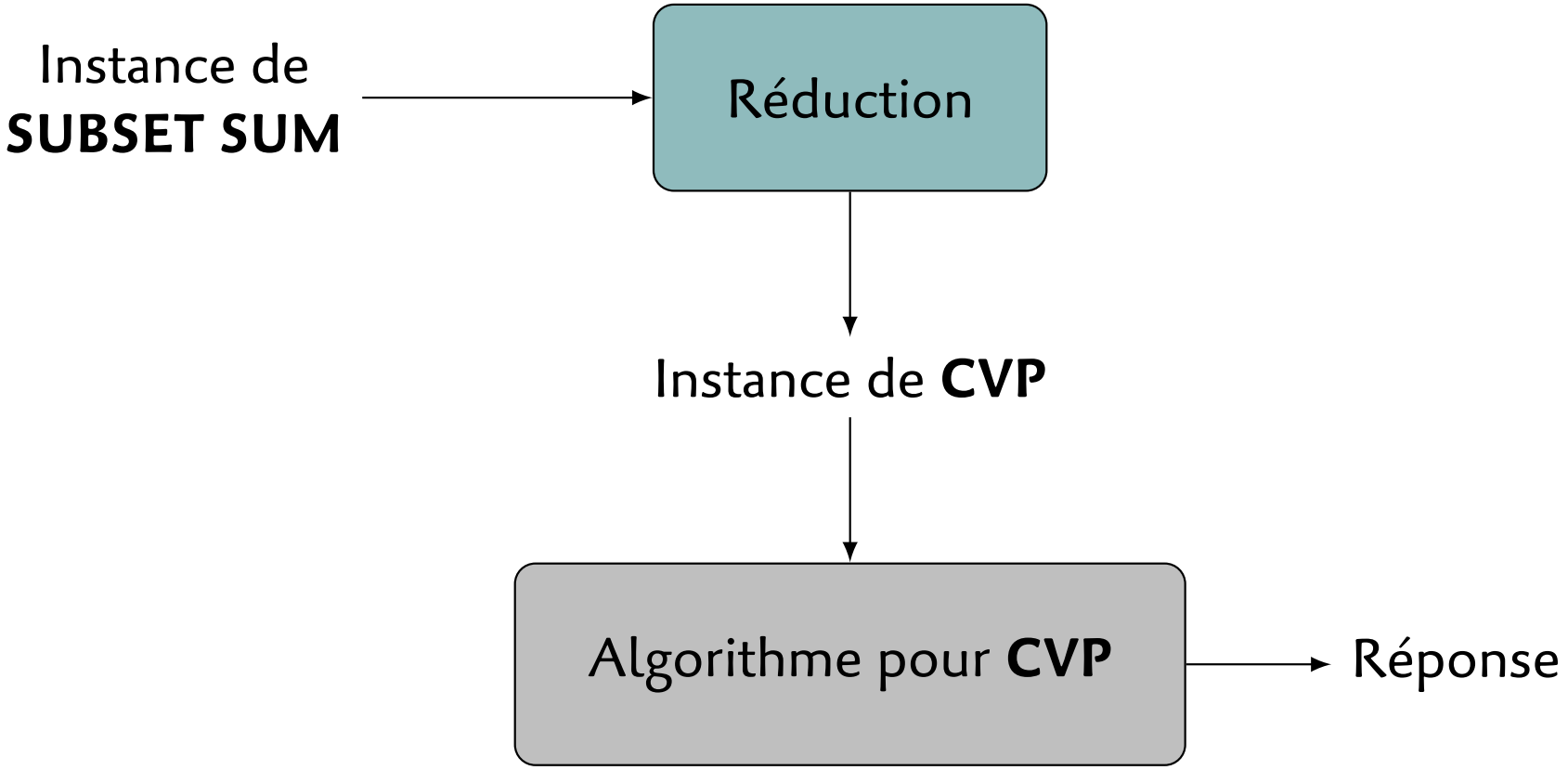
Closest Vector Problem (CVP) — version décisionnelle

Étant donné une base d'un réseau $\mathcal{L}$, un vecteur $\mathbf{y}$ et une distance d, déterminer s'il existe $\mathbf{x} \in \mathcal{L}$ tel que $\|\mathbf{y} - \mathbf{x}\| \leq d$

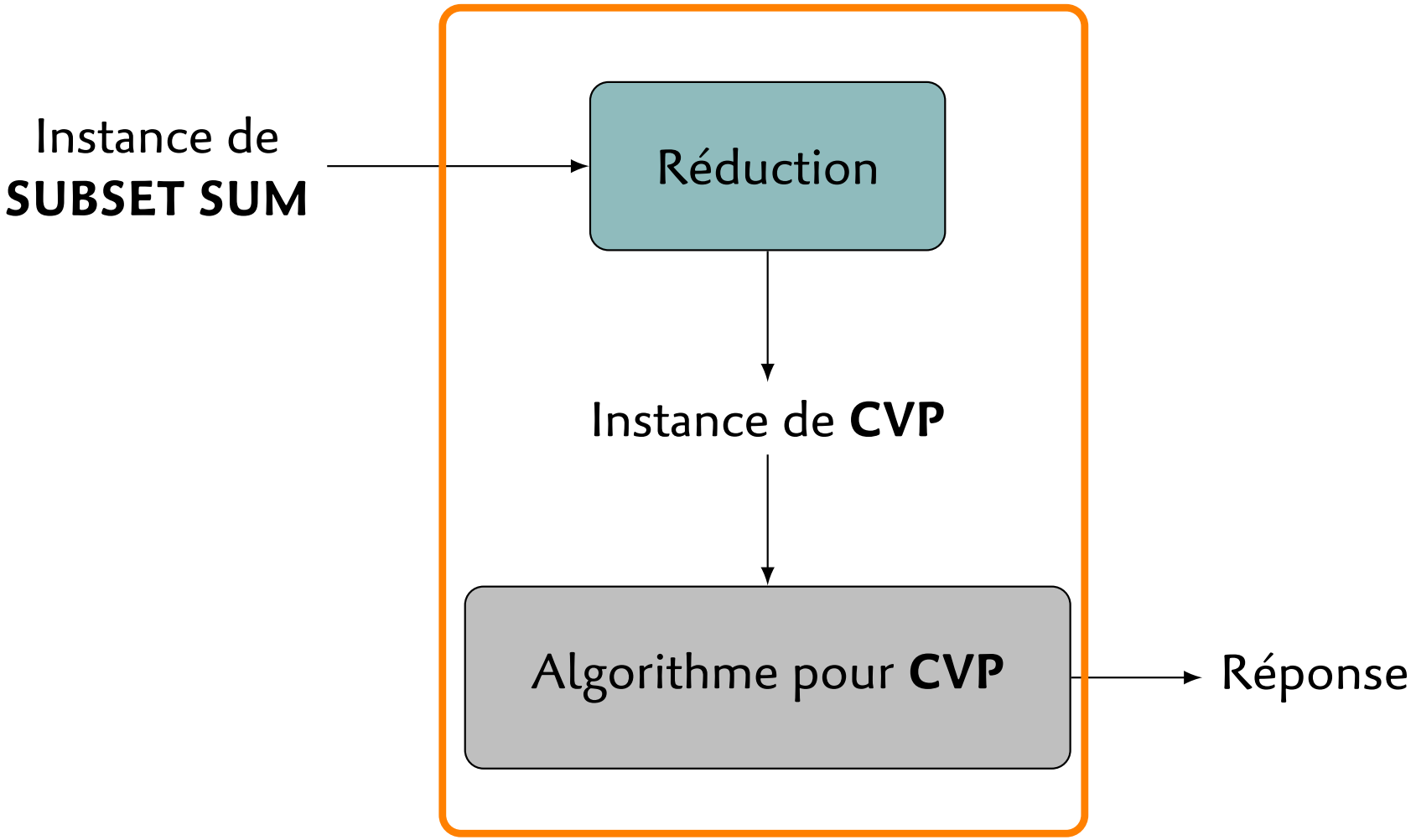
Theorem

CVP est ***NP-dur***

NP-difficulté de CVP : Stratégie



NP-difficulté de CVP : Stratégie



NP-difficulté de CVP : preuve

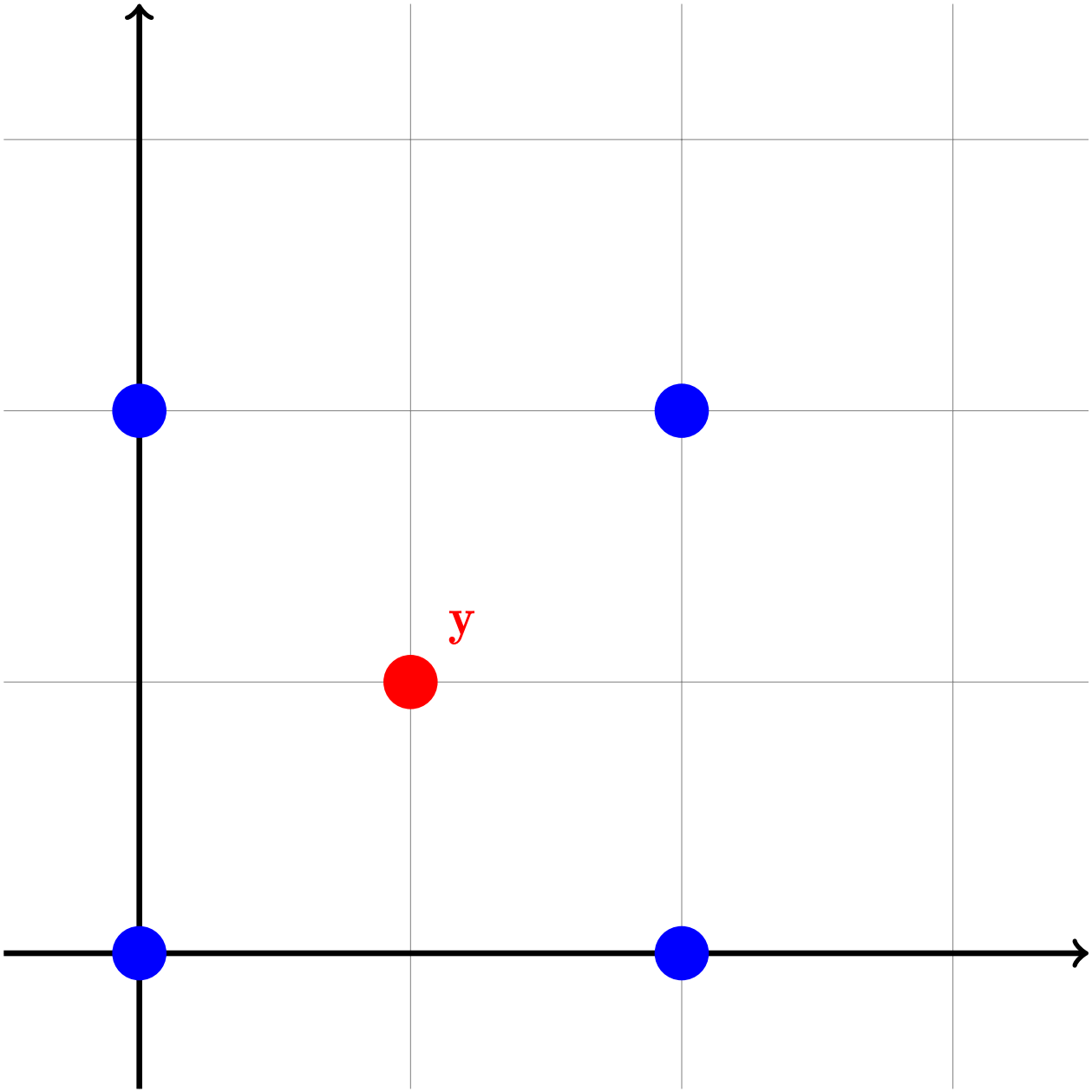
Démonstration.

- Réduction : CVP $\to$ SUBSET SUM $\to$ VERTEX COVER
- On part d'une instance $(A_1, \ldots, A_n), t$ de SUBSET SUM
  - Existe-t-il $\mathbf{x} \in \{0,1\}^n$ tel que $\sum x_i A_i = t$?
- On considère un réseau spécial :

$$B = \begin{pmatrix} A_1 & 2 & & & & \\ A_2 & & 2 & & & \\ A_3 & & & 2 & & \\ \vdots & & & & \ddots & \\ A_n & & & & & 2 \end{pmatrix}$$

- Si $\sum x_i A_i = t$, alors $xB = (t, 2x_1, 2x_2, 2x_3, \ldots, 2x_n)$
- xB est proche de $\mathbf{y} = (t, 1, 1, \ldots, 1)$. Distance $= \sqrt{n}$

NP-difficulté de CVP : preuve (illustration en 2D)



NP-difficulté de CVP : preuve (suite)

Démonstration.

$$B = \begin{pmatrix} A_1 & 2 & & & \\ A_2 & & 2 & & \\ A_3 & & & 2 & \\ \vdots & & & & \ddots \\ A_n & & & & & 2 \end{pmatrix}$$

$$xB = \left(\sum_{i=1}^{n} x_i A_i, 2x_1, 2x_2, 2x_3, \ldots, 2x_n \right)$$

- Décider CVP pour B, $\mathbf{y} = (t, 1, 1, \ldots, 1)$, distance $\sqrt{n}$
  - Ça va révéler la solution du SUBSET SUM de départ
- Si $\sum x_i A_i = t$, $\|\mathbf{x}B - \mathbf{y}\| = \sqrt{n}$
- Si $z \in L$, n dernières coordonnées paires $\Rightarrow \|\mathbf{y} - \mathbf{z}\| \geq \sqrt{n}$
- $\|\mathbf{y} - \mathbf{z}\| = \sqrt{n} \Longrightarrow z_1 = t$ et $z_i \in \{0, 2\}$ pour $i \geq 2$
  - $\mathbf{z}$ décrit une solution valide au SUBSET SUM de départ

□

Les classiques

CVP / SVP **exact**

- Complexité : $\mathcal{O}\left(2^{d \log d}\right)$ (méthodes d'énumération)
- Facile en petite dimension (≤ 50 disons)
  - Algorithmes disponibles dans `fpylll` ou `G6K`
  - On va les voir ici même !
- Impossible en grande dimension (≥ 300 disons)

CVP / SVP **approché** en grande dimension

- « Facile » d'obtenir γ exponentiel en la dimension
  - Algorithmes disponibles dans `fpylll`, `SageMath`, etc.
- Difficile d'obtenir γ faible

Plan

Teasing : sécurité de RSA avec Petits exposants secrets

Introduction aux réseaux euclidiens

Propriétés de base, plus court vecteur, volume

Problèmes algorithmiques dans les réseaux

Algorithmes de « réduction de réseau »
- En dimension 2 : réduction de Lagrange
- Algorithmes basés sur l'orthogonalisation
  - Orthogonalisation de Gram-Schmidt
  - La size-reduction

La réduction de réseau

- Construire une « bonne » base à partir d'une base pourrie

Pourquoi faire ?

- Cryptanalyse
- Comprendre à quel point le problème est dur...
- ... pour l'utiliser comme « hypothèse de sécurité »

Utilisation typique des réseaux en cryptanalyse

Procédure habituelle #1

1. Infos publiques $\rightsquigarrow$ base (pourrie) d'un réseau
2. vecteur spécial $\mathbf{x}$ est « anormalement court »
3. Résoudre (approx-)SVP $\rightsquigarrow$ obtenir $\mathbf{x}$
4. Extraire secrets de $\mathbf{x}$

Procédure habituelle #2

1. Infos publiques $\rightsquigarrow$ base (pourrie) d'un réseau et $\mathbf{y}$
2. vecteur spécial $\mathbf{x}$ est « anormalement proche » de $\mathbf{y}$
3. Résoudre (approx-)CVP sur $\mathbf{y}$ $\rightsquigarrow$ obtenir $\mathbf{x}$
4. Extraire secrets de $\mathbf{x}$

La réduction de réseau

- Construire une « bonne » base à partir d'une base pourrie

Pourquoi faire ?

- Cryptanalyse
- Comprendre à quel point le problème est dur...
- ... pour l'utiliser comme « hypothèse de sécurité »

Qu'est-ce qu'une « bonne »base ?

- Permet de résoudre (approx-)SVP et (approx-)CVP en gros
- Quel critère optimiser ?
  - Vecteurs courts ?
  - Défaut d'orthogonalité ?

Quel est l'objectif ?

$$B := \begin{pmatrix} 2 & & & & \\ & 2 & & & \\ & & 2 & & \\ & & & 2 & \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

- $(-1,-1,-1,-1,2)B = (0,0,0,0,2)$ est de norme 2
- Donc les 5 vecteurs les plus courts :

$$B' := \begin{pmatrix} 2 & & & & \\ & 2 & & & \\ & & 2 & & \\ & & & 2 & \\ & & & & 2 \end{pmatrix}$$

  n'engendrent pas le même réseau !
- Une base réduite peut toujours contenir LE plus court
- Les d plus courts vecteurs ne sont pas forcément une base

Plan

Teasing : sécurité de RSA avec Petits exposants secrets

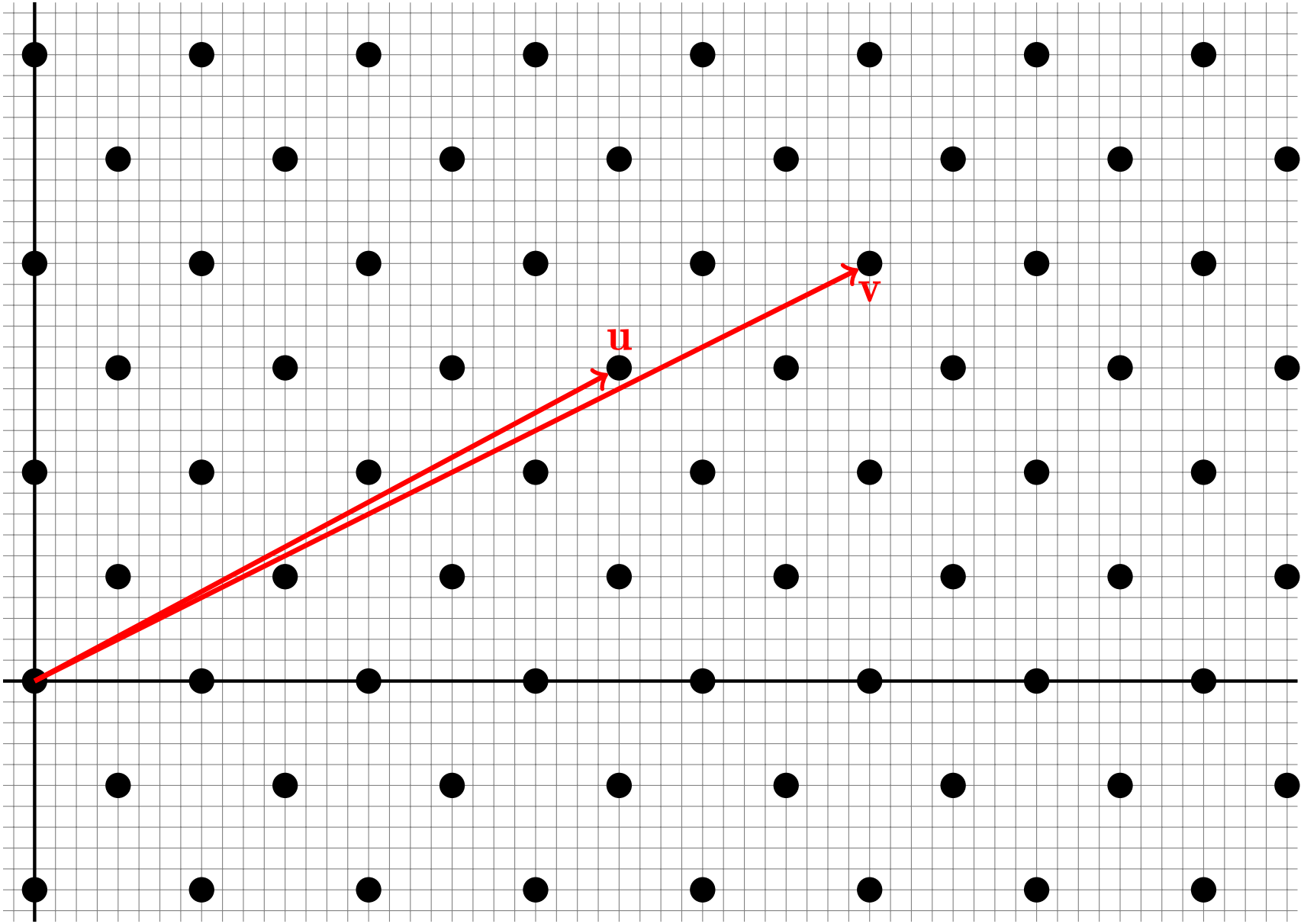
Introduction aux réseaux euclidiens

Propriétés de base, plus court vecteur, volume

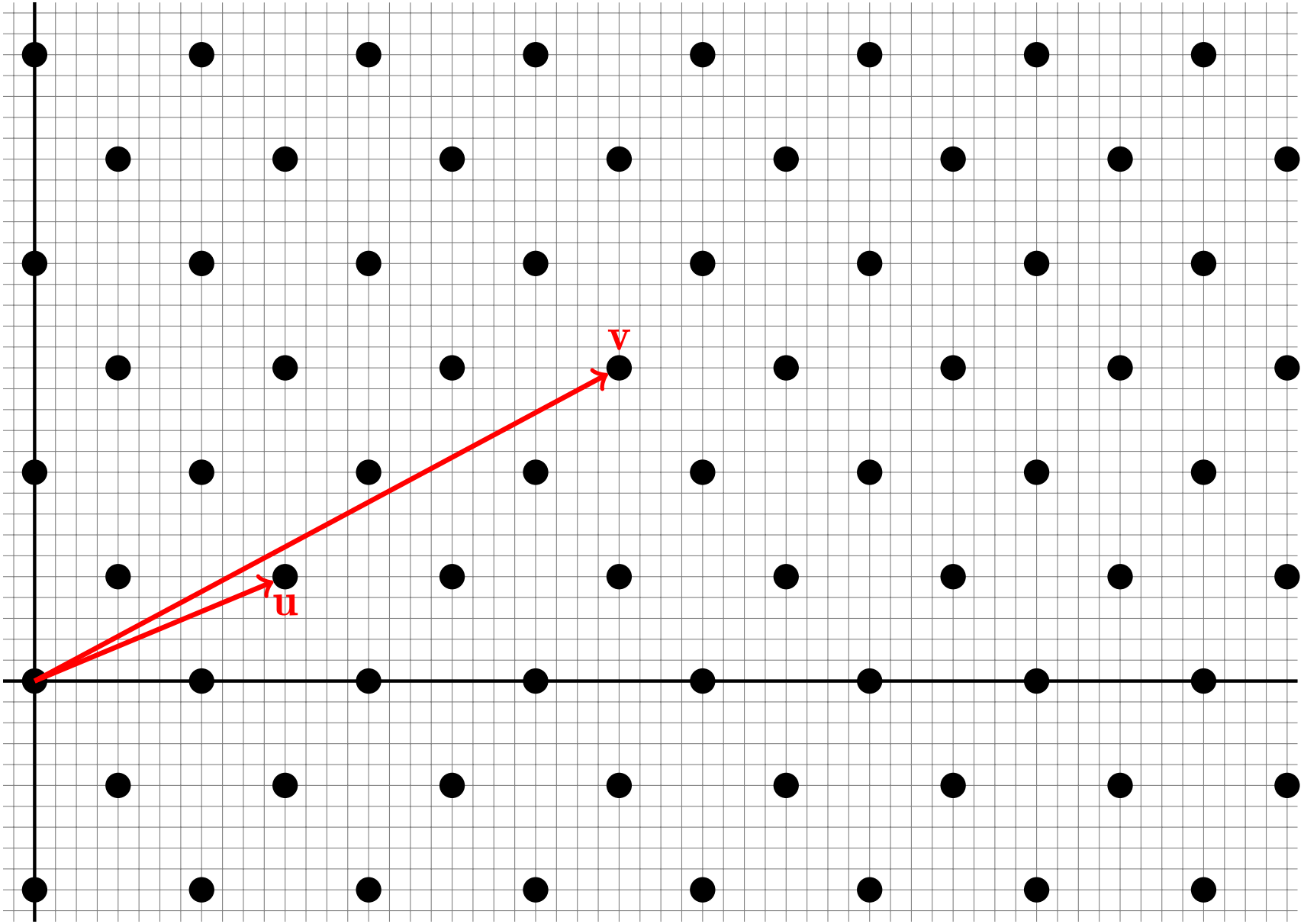
Problèmes algorithmiques dans les réseaux

Algorithmes de « réduction de réseau »
- En dimension 2 : réduction de Lagrange
- Algorithmes basés sur l'orthogonalisation
  - Orthogonalisation de Gram-Schmidt
  - La size-reduction

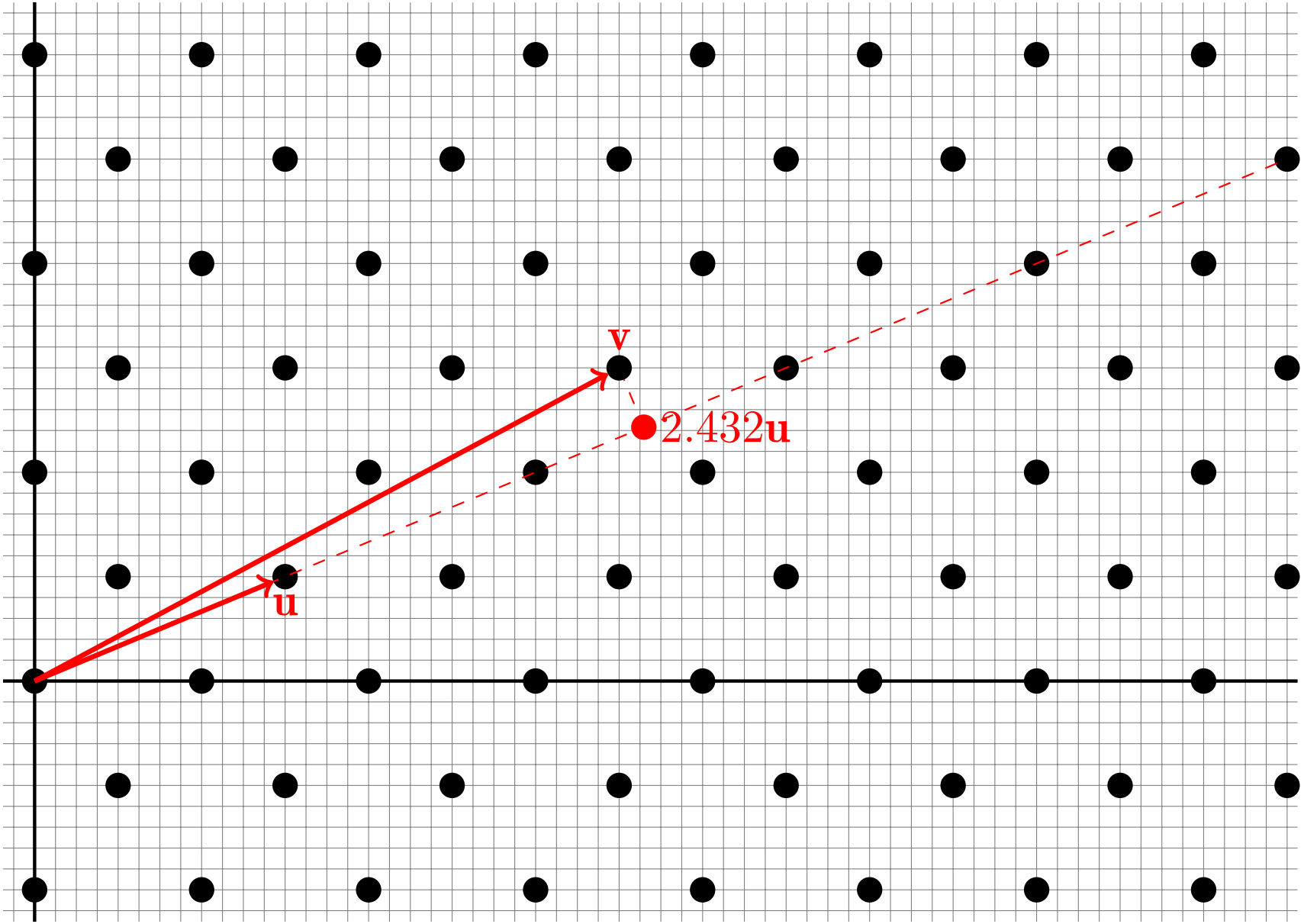
La « réduction de réseau »



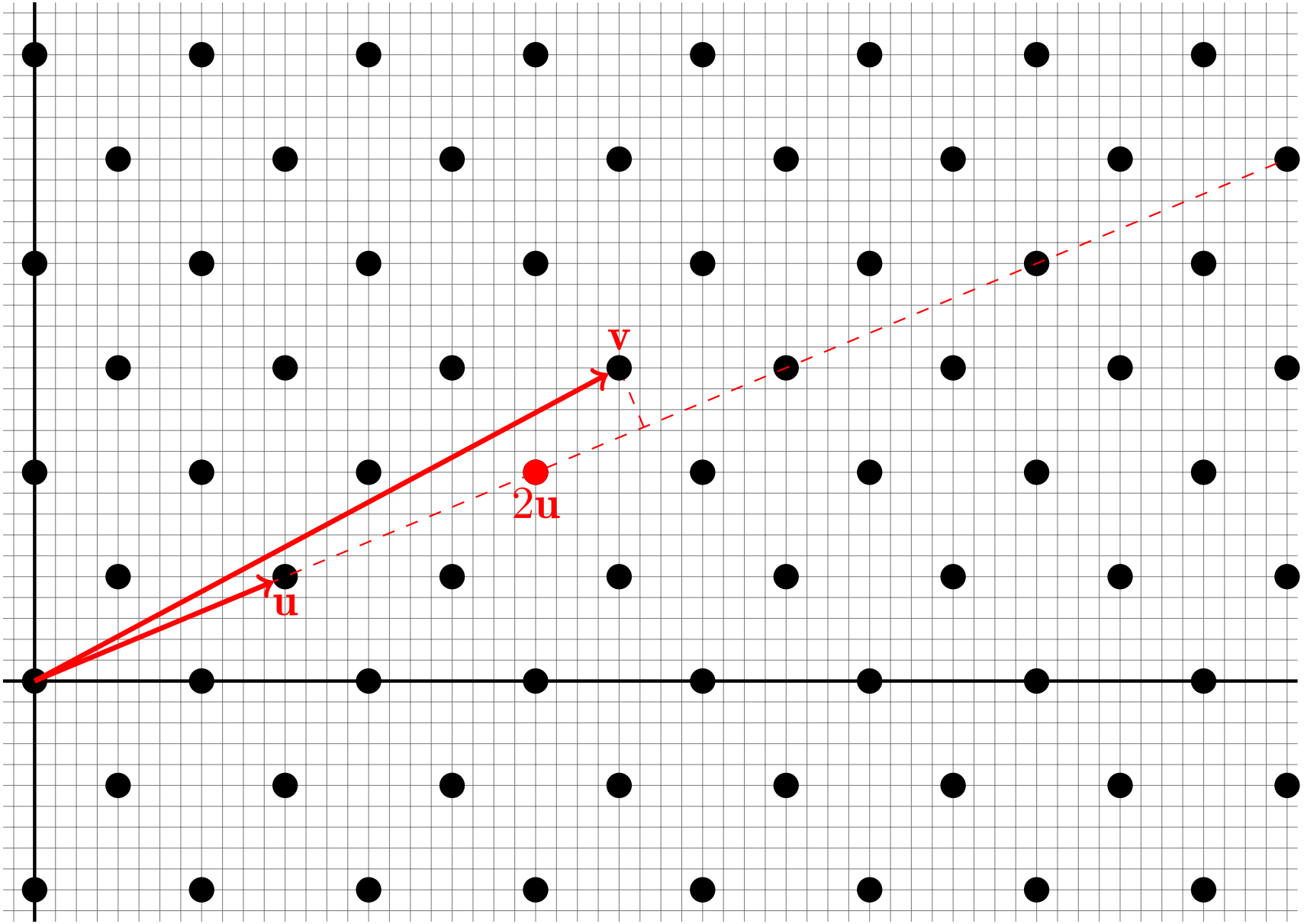
La « réduction de réseau »



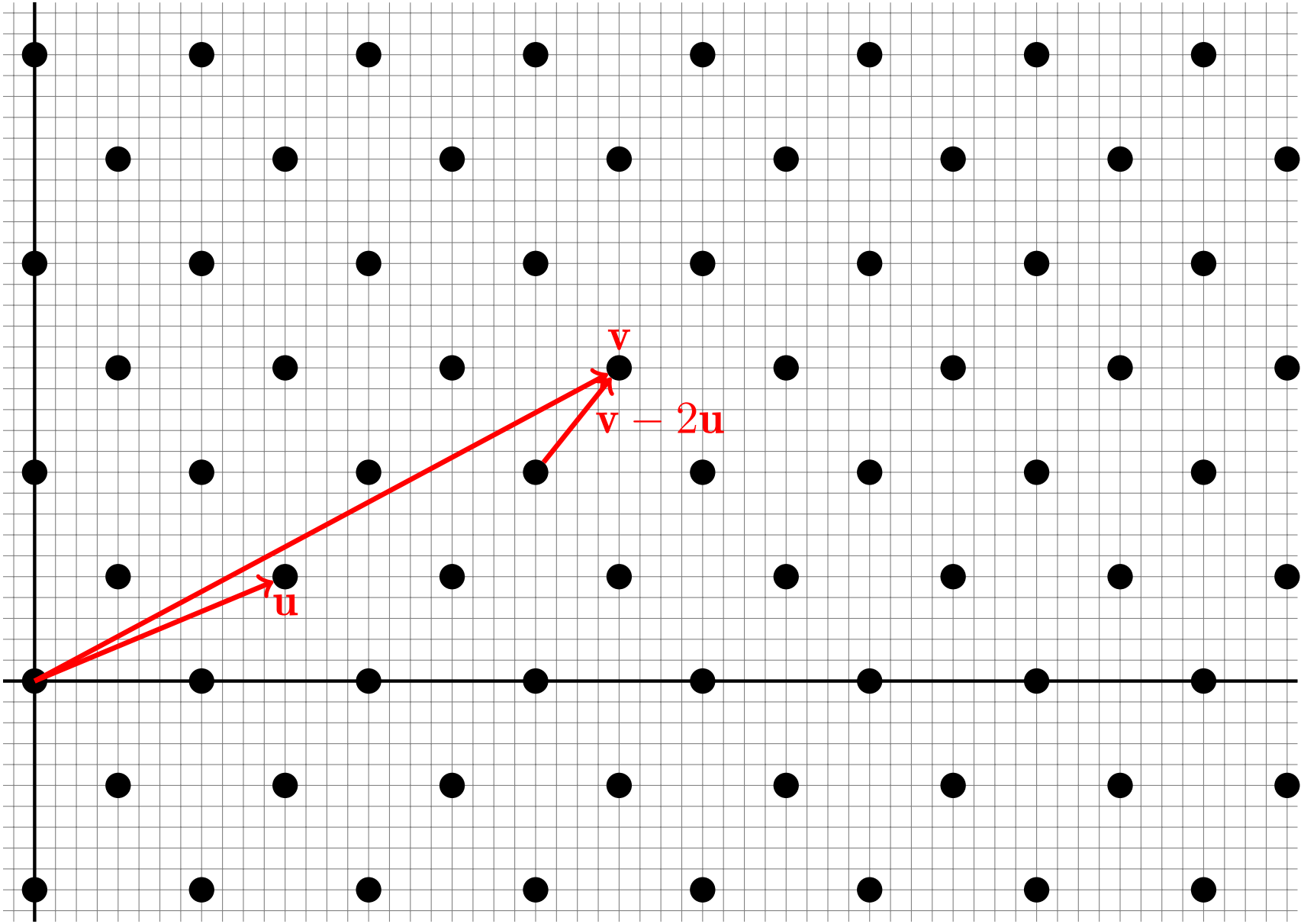
La « réduction de réseau »



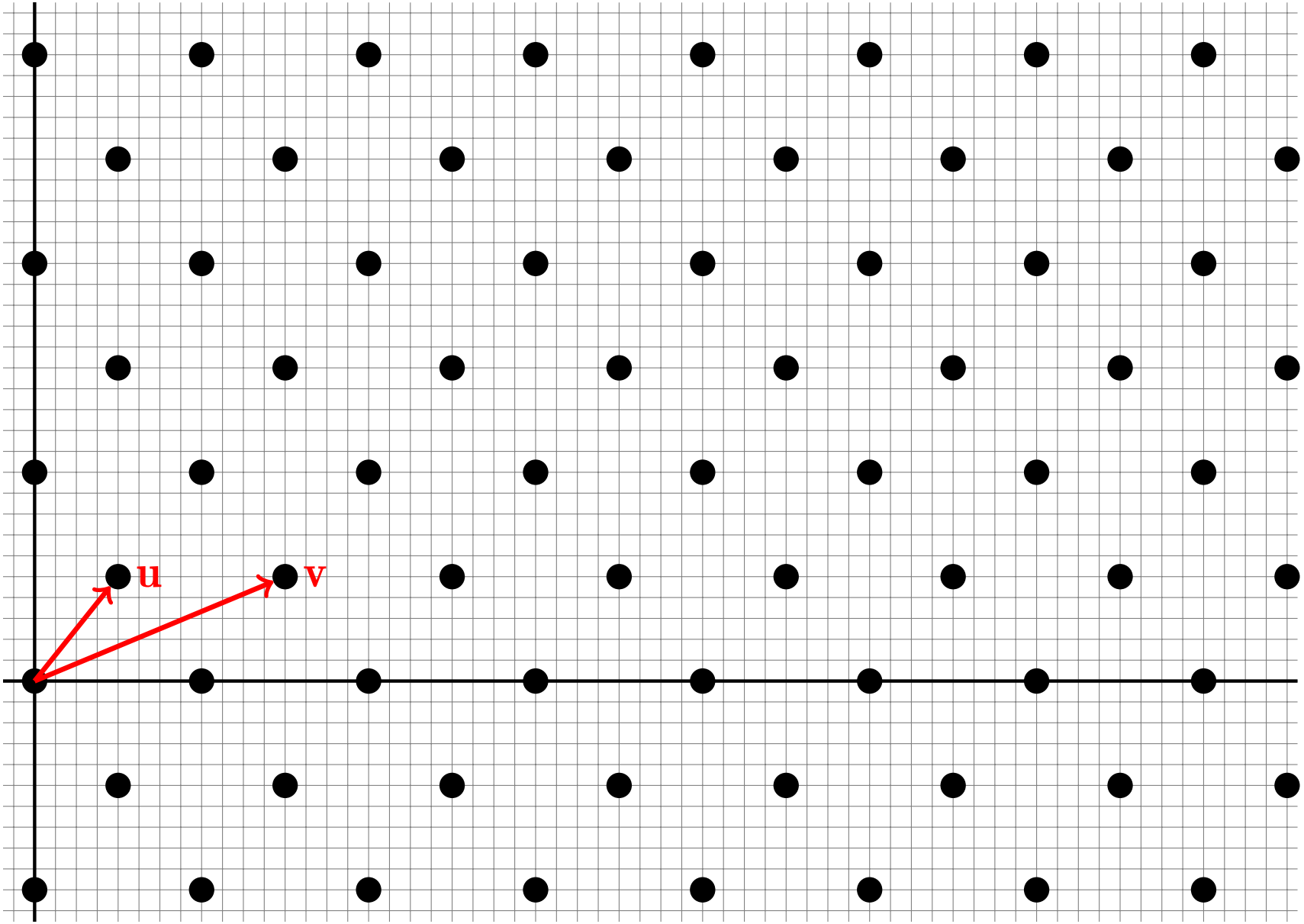
La « réduction de réseau »



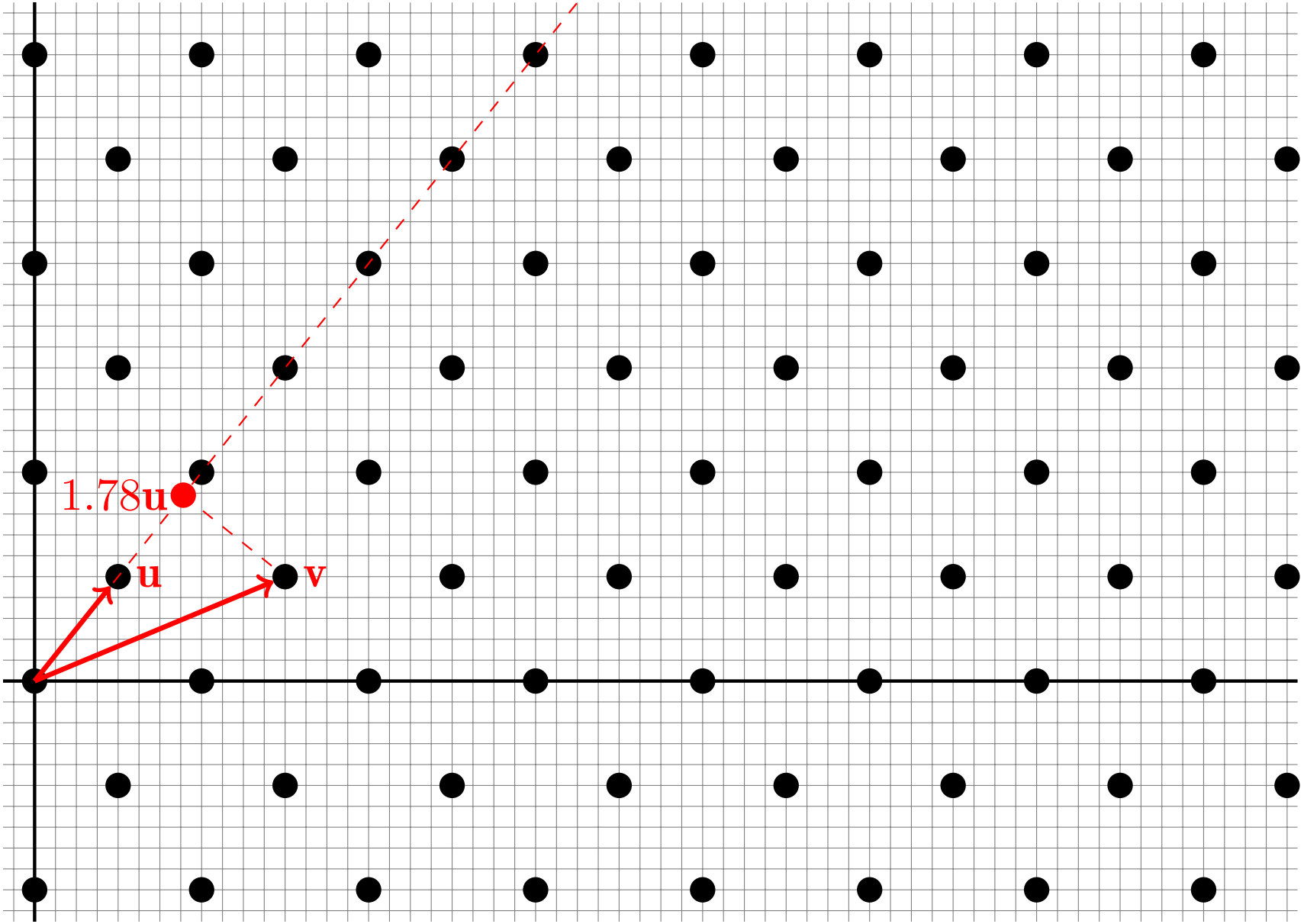
La « réduction de réseau »



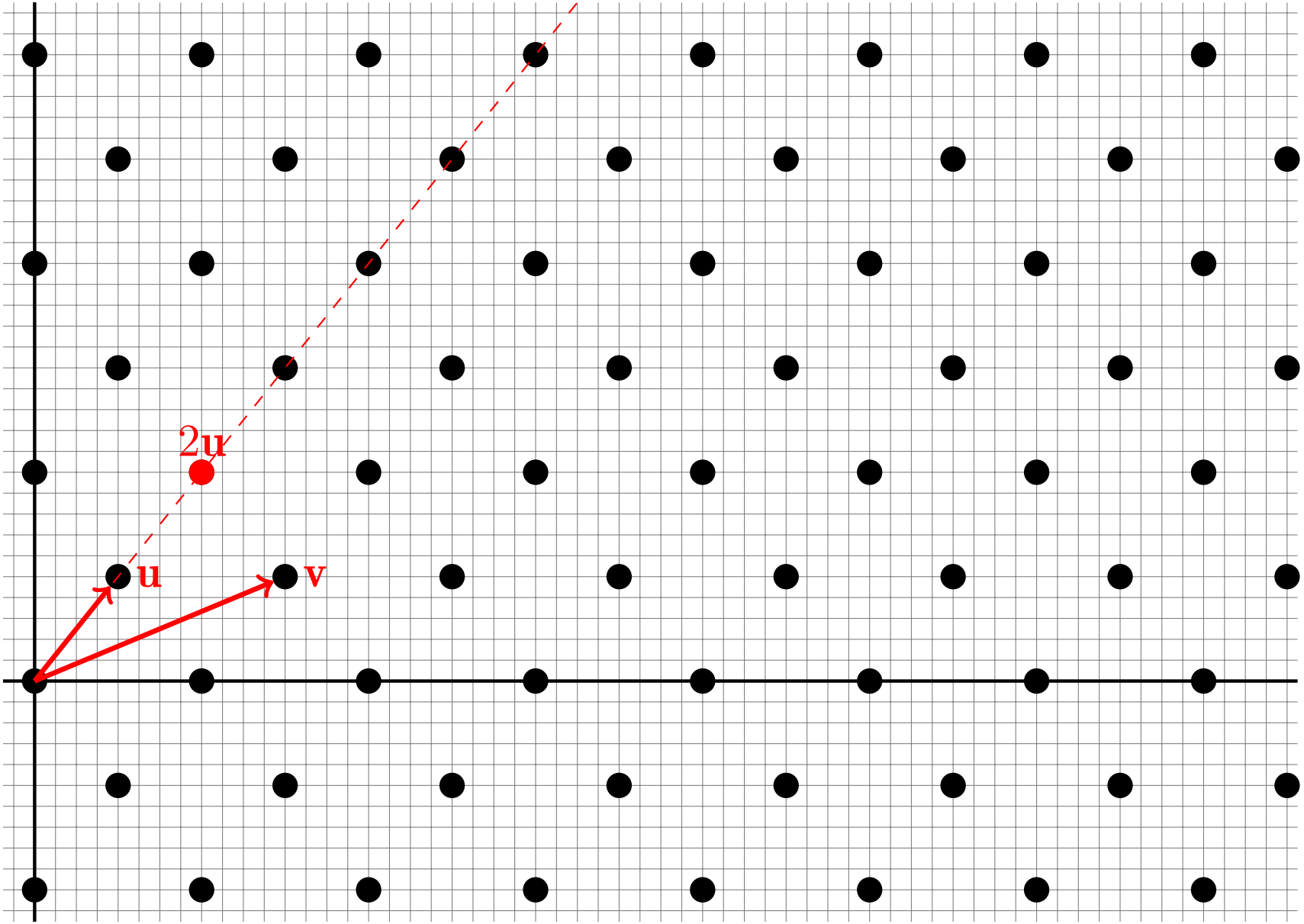
La « réduction de réseau »



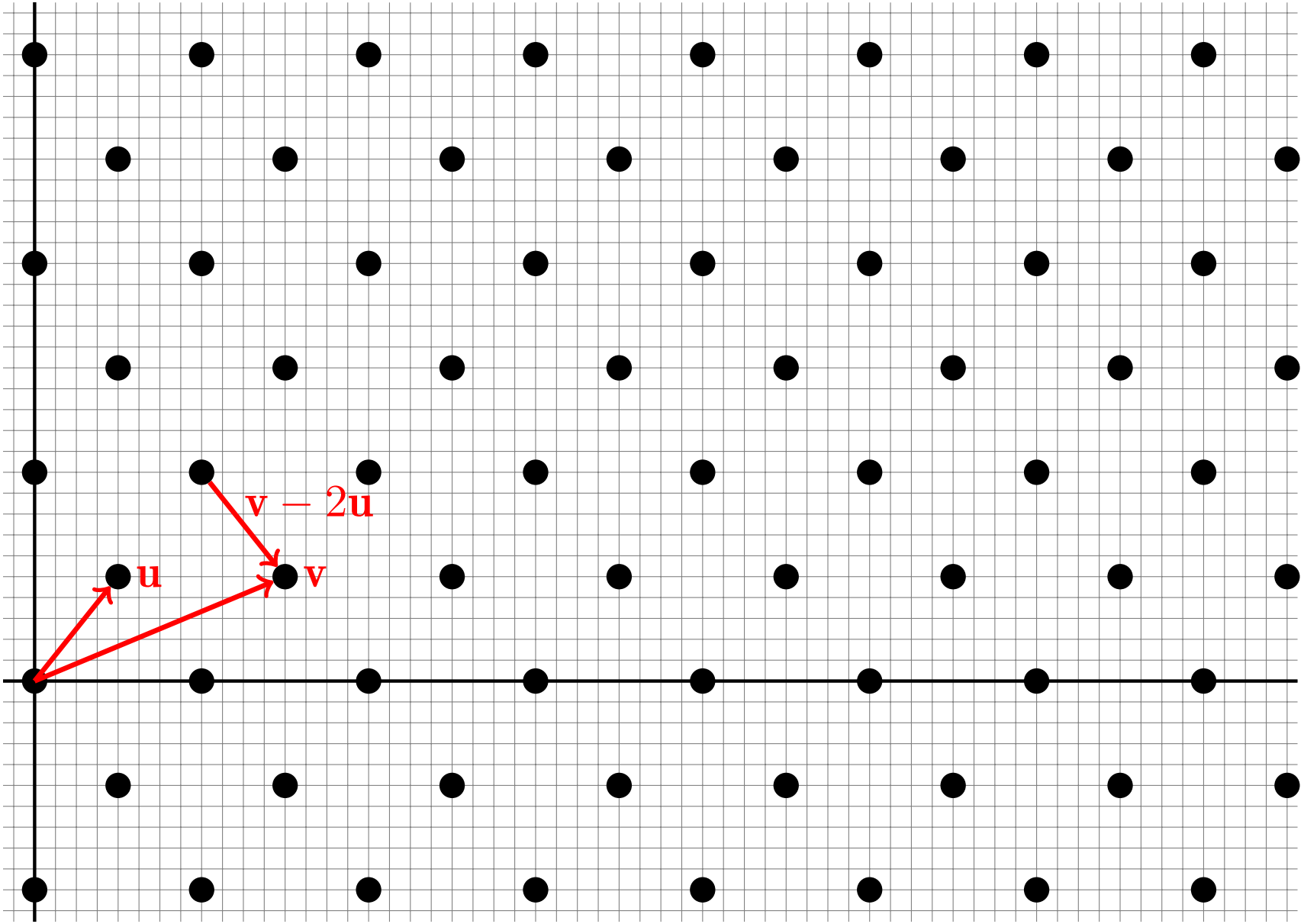
La « réduction de réseau »



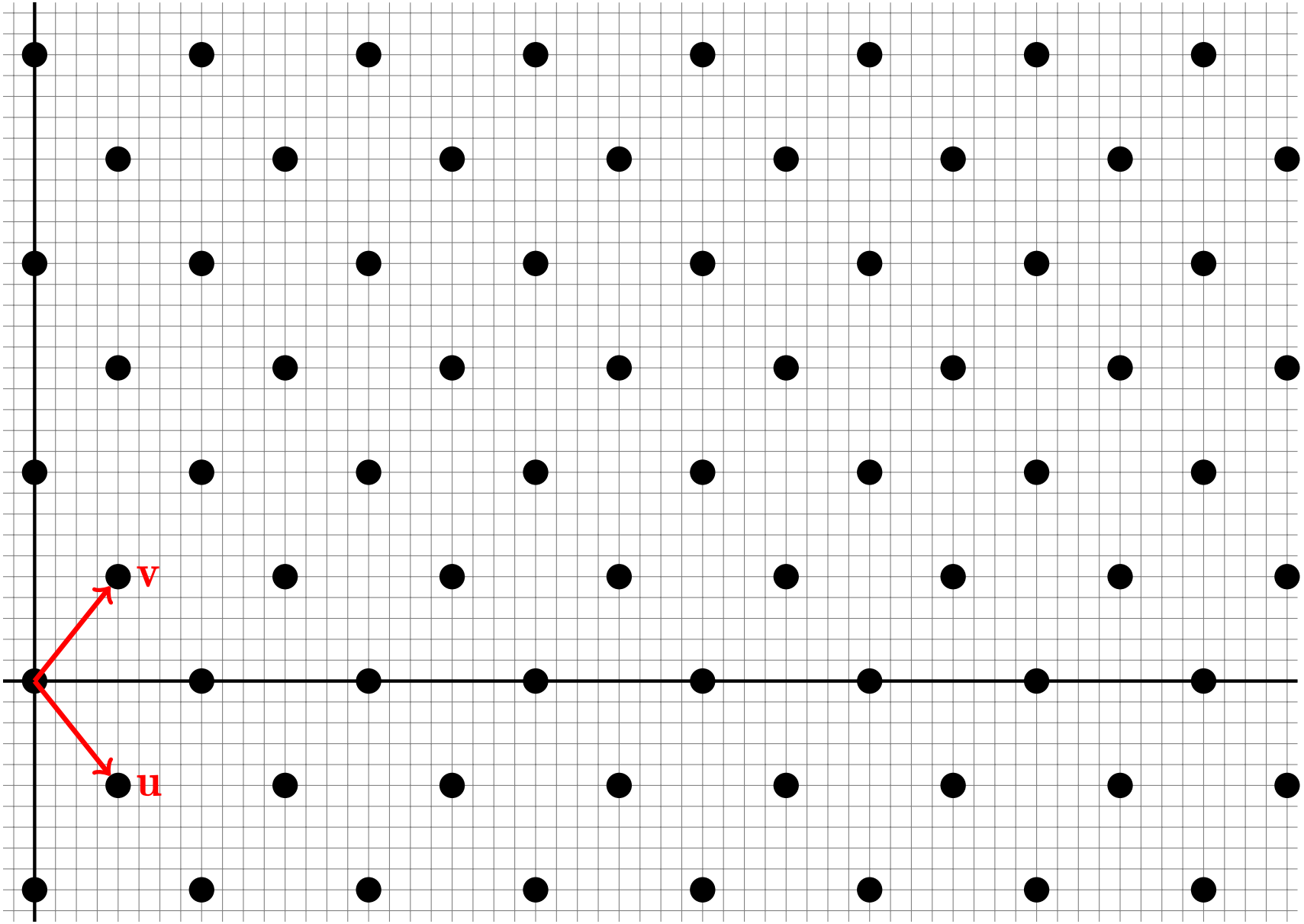
La « réduction de réseau »



La « réduction de réseau »



La « réduction de réseau »



Algorithme de réduction de Lagrange (1736–1813)

```
function LagrangeReduce(u, v)
    repeat                                  ▷ Cette version est cubique
        x ← ⟨u, v⟩ / ‖u‖²                   ▷ ⟨u, u⟩ = ‖u‖²
        w ← v − ⌈x⌋u
        v ← u
        u ← w
    until ‖u‖ ≥ ‖v‖
    return (v, u)                           ▷ ‖v‖ ≤ ‖u‖ et |⟨v, u⟩| ≤ ‖v‖²/2
end function
```

Theorem

Si $(\mathbf{r}, \mathbf{s}) \leftarrow$ *LagrangeReduce*$(\mathbf{u}, \mathbf{v})$*, alors :*

- *Il y a des implantations qui fonctionnent en temps quadratique*
- $\mathbf{r}$ *est le plus court vecteur du réseau*
- $\mathbf{s}$ *est le 2ème plus court vecteur du réseau*
- $\|\mathbf{r}\| \times \|\mathbf{s}\| \leq \sqrt{4/3} \times V$ *(V est le volume du réseau)*

Plan

Teasing : sécurité de RSA avec Petits exposants secrets

Introduction aux réseaux euclidiens

Propriétés de base, plus court vecteur, volume

Problèmes algorithmiques dans les réseaux

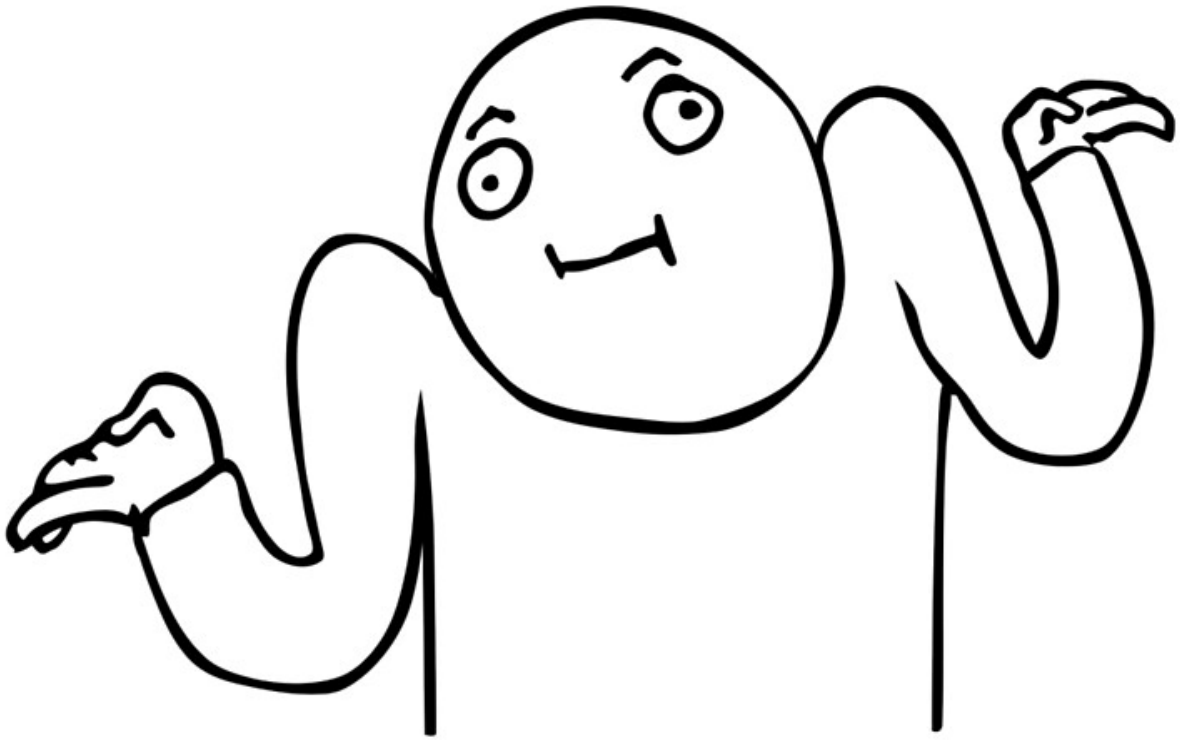
Algorithmes de « réduction de réseau »
- En dimension 2 : réduction de Lagrange
- Algorithmes basés sur l'orthogonalisation
  - Orthogonalisation de Gram-Schmidt
  - La size-reduction

Réduction de réseau

- Le cas $d = 2$ est reglé par l'algorithme de Lagrange
  - Renvoie les deux vecteurs les plus courts
- Et $d \geq 3$?

Réduction de réseau

- Le cas $d = 2$ est reglé par l'algorithme de Lagrange
  - Renvoie les deux vecteurs les plus courts
- Et $d \geq 3$?



Mauvaise idée : généraliser Lagrange en plus grande dimension

Même idée générale :

Pour $0 \leq i < n$:

- Projeter orthogonalement $\mathbf{b}_i$ sur $\langle \mathbf{b}_1, \ldots, \mathbf{b}_{i-1} \rangle$
- Trouver le point $\mathbf{x}$ du sous-réseau engendré par $(\mathbf{b}_1, \ldots, \mathbf{b}_{i-1})$ le plus proche de la projection de $\mathbf{b}_i$
- Remplacer $\mathbf{b}_i$ par $\mathbf{b}_i - \mathbf{x}$

Mauvaise idée : généraliser Lagrange en plus grande dimension

Même idée générale :

Pour $0 \leq i < n$:

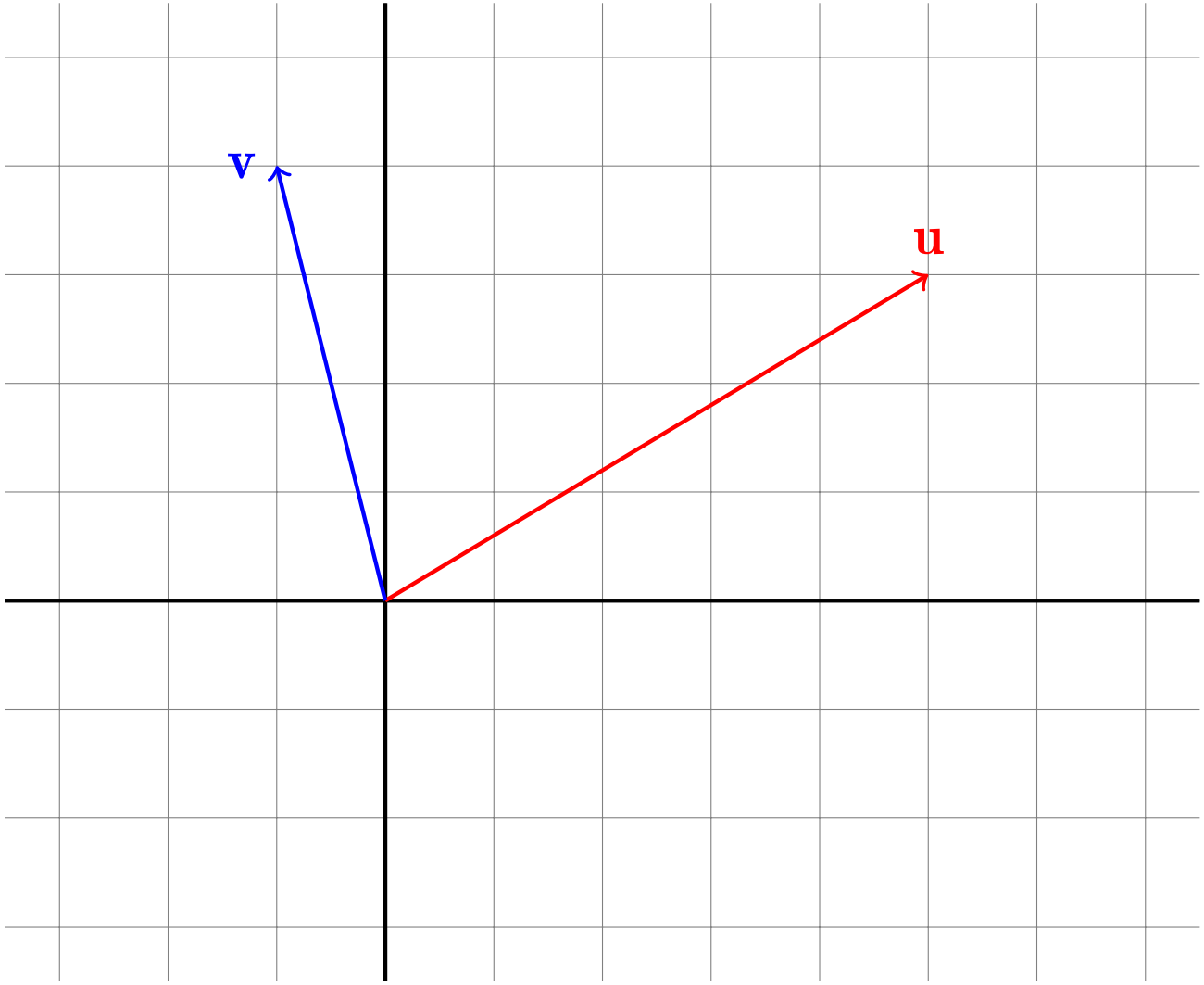
- Projeter orthogonalement $\mathbf{b}_i$ sur $\langle \mathbf{b}_1, \ldots, \mathbf{b}_{i-1} \rangle$
- Trouver le point $\mathbf{x}$ du sous-réseau engendré par $(\mathbf{b}_1, \ldots, \mathbf{b}_{i-1})$ le plus proche de la projection de $\mathbf{b}_i$
- Remplacer $\mathbf{b}_i$ par $\mathbf{b}_i - \mathbf{x}$

Oooops

Et comment on résout l'instance de Closest Vector Problem ?

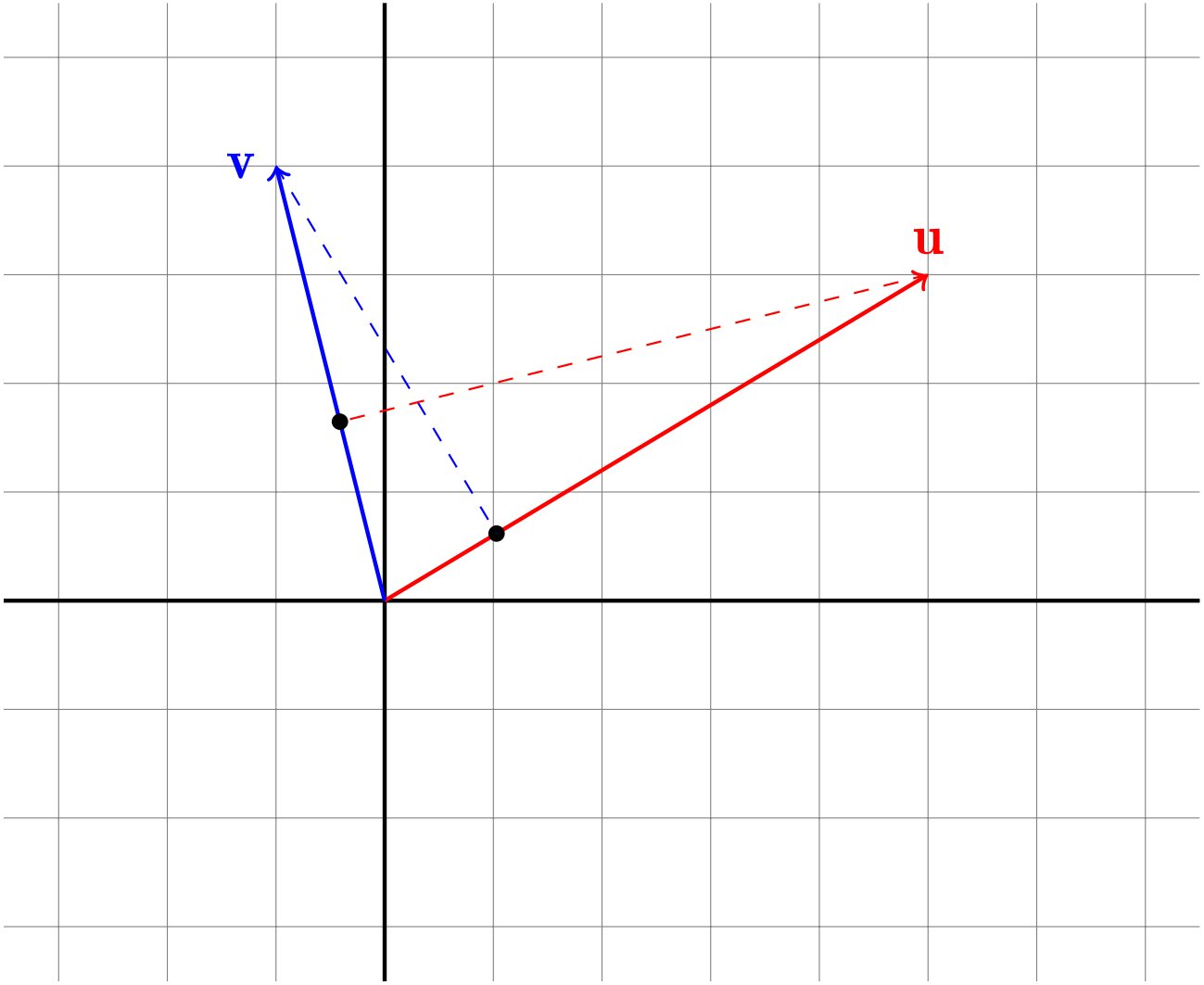
La « size-reduction »

Il y avait quand même quelque chose à garder chez Lagrange…



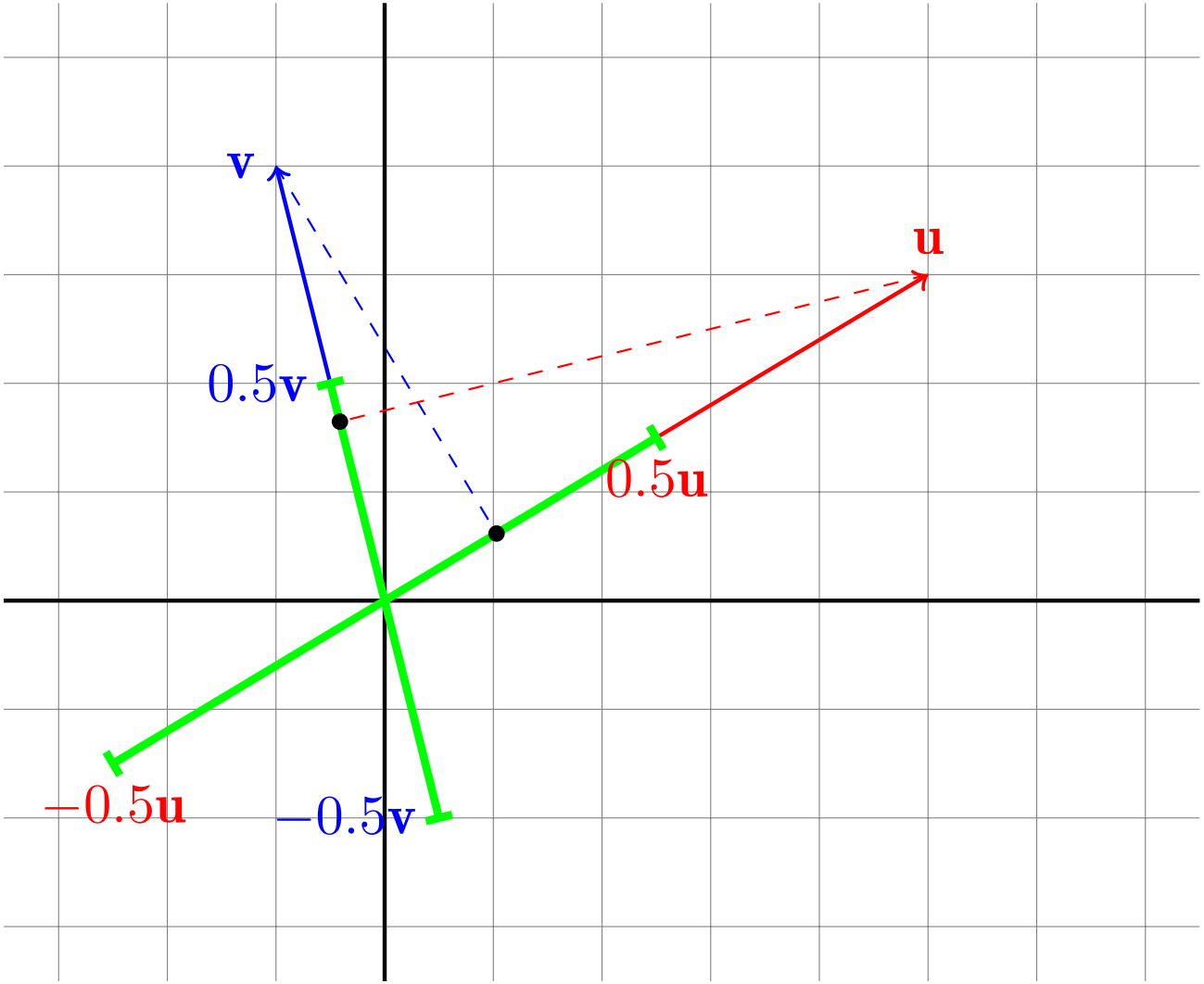
La « size-reduction »

Il y avait quand même quelque chose à garder chez Lagrange...

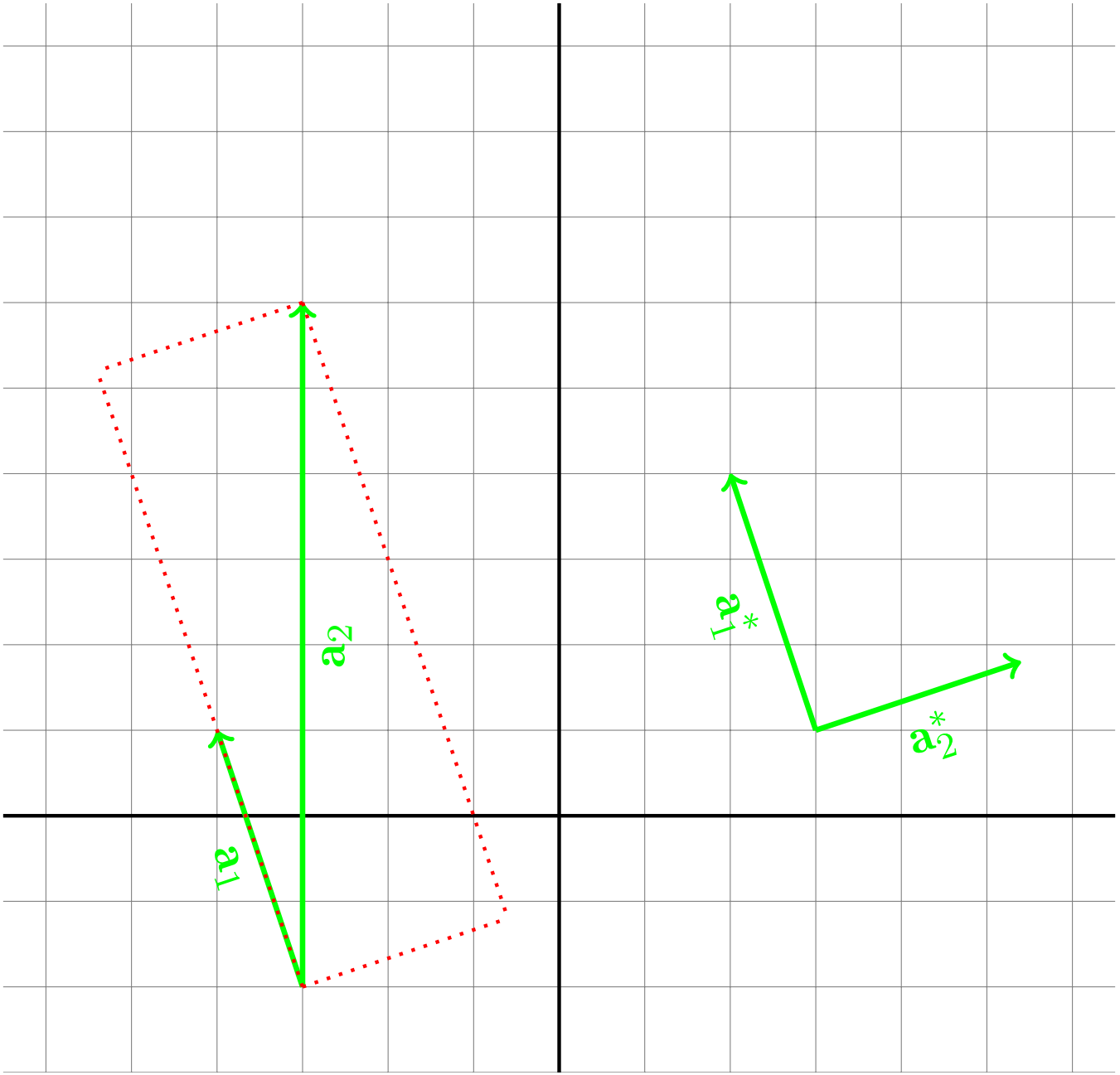


La « size-reduction »

Il y avait quand même quelque chose à garder chez Lagrange...



Orthogonalisation de Gram-Schmidt



Procédé d'orthogonalisation de Gram-Schmidt

ENTRÉE $(\mathbf{b}_1, \ldots, \mathbf{b}_d) \subseteq \mathbb{R}^n$ (**linéairement indépendants**)

SORTIE $(\mathbf{b}_1^*, \ldots, \mathbf{b}_d^*)$ tels que :

- $\mathrm{Span}(\mathbf{b}_1, \ldots, \mathbf{b}_i) = \mathrm{Span}(\mathbf{b}_1^*, \ldots, \mathbf{b}_i^*)$ $(1 \leq i \leq d)$
- $\langle \mathbf{b}_i^*, \mathbf{b}_j^* \rangle = 0$ pour tout $i \neq j$

Idée maîtresse

$$\mathbf{b}_i^* = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{ij} \mathbf{b}_j^*$$

- $\langle \mathbf{b}_i^*, \mathbf{b}_k^* \rangle = \langle \mathbf{b}_i, \mathbf{b}_k^* \rangle - \mu_{ik} \langle \mathbf{b}_k^*, \mathbf{b}_k^* \rangle = 0$
  $\Rightarrow \mu_{ik} = \langle \mathbf{b}_i, \mathbf{b}_k^* \rangle \,/\, \langle \mathbf{b}_k^*, \mathbf{b}_k^* \rangle$ (easy)
- $\sum_{j=1}^{i-1} \mu_{ij} \mathbf{b}_j^*$: projection de $\mathbf{b}_i$ sur $\mathrm{Span}(\mathbf{b}_1, \ldots, \mathbf{b}_{i-1})$
  - $\|\mathbf{b}_i^*\|$ = distance de $\mathbf{b}_i$ à $\mathrm{Span}(\mathbf{b}_1, \ldots, \mathbf{b}_{i-1})$

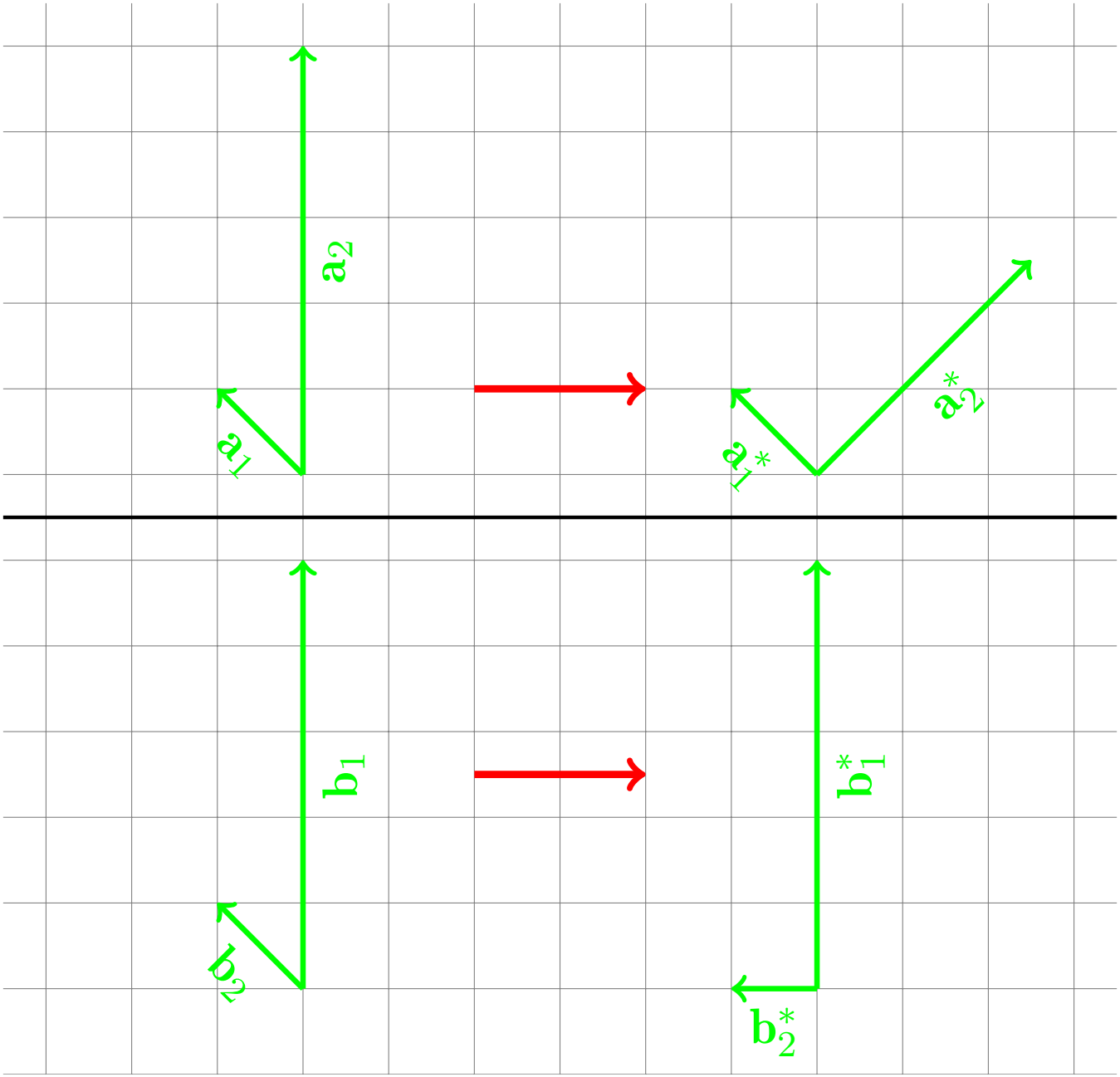
Orthogonalisation de Gram-Schmidt (suite)

$$\underbrace{\begin{pmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \vdots \\ \mathbf{b}^{d-1} \\ \mathbf{b}_d \end{pmatrix}}_{B} = \underbrace{\begin{pmatrix} 1 & & & & \\ \mu_{21} & 1 & & & \\ \vdots & & \ddots & & \\ \vdots & & & \ddots & \\ \mu_{d1} & \mu_{d,2} & \cdots & \mu_{d,d-1} & 1 \end{pmatrix}}_{\mu} \underbrace{\begin{pmatrix} \mathbf{b}_1^* \\ \mathbf{b}_2^* \\ \vdots \\ \mathbf{b}_{d-1}^* \\ \mathbf{b}_d^* \end{pmatrix}}_{B^*}$$

Complexité

- Attention, les coordonnées de $\mathbf{b_i}^*$ et les μ_{ij} sont des rationnels qui grossissent avec i
- Tout peut s'écrire avec dénominateur $(\mathrm{Vol}(\mathbf{b}_1, \ldots, \mathbf{b}_d))^2$
- nd^2 opérations sur des nombre de $\approx 2\log_2 V$ bits...

Orthogonalisation de Gram-Schmidt : l'ordre compte



Vers la « size-reduction »

$$M = \begin{pmatrix} 782 & 550 & 745 & 851 & 167 \\ 765 & 868 & 487 & 524 & 361 \\ 870 & 86 & 982 & 901 & 594 \\ 153 & 978 & 13 & 576 & 421 \\ 975 & 136 & 960 & 281 & 533 \end{pmatrix}$$

$$M = \begin{pmatrix} 1 & & & & \\ 0.8755 & 1 & & & \\ 1.0468 & -0.4701 & 1 & & \\ 0.5527 & 1.2627 & -0.3044 & 1 & \\ 0.8466 & 0.0884 & 1.0443 & -0.7971 & 1 \end{pmatrix} \times B^*$$

Normes = (1490.3, 1407.3, 1700.9, 1220.2, 1501.2)

Vers la « size-reduction »

$$M = \begin{pmatrix} 782 & 550 & 745 & 851 & 167 \\ -17 & 318 & -258 & -327 & 194 \\ 870 & 86 & 982 & 901 & 594 \\ 153 & 978 & 13 & 576 & 421 \\ 975 & 136 & 960 & 281 & 533 \end{pmatrix}$$

$$M = \begin{pmatrix} 1 & & & & \\ -0.1244 & 1 & & & \\ 1.0468 & -0.4701 & 1 & & \\ 0.5527 & 1.2627 & -0.3044 & 1 & \\ 0.8466 & 0.0884 & 1.0443 & -0.7971 & 1 \end{pmatrix} \times B^*$$

Normes = (1490.3, 559.0, 1700.9, 1220.2, 1501.2)

Vers la « size-reduction »

$$M = \begin{pmatrix} 782 & 550 & 745 & 851 & 167 \\ -17 & 318 & -258 & -327 & 194 \\ 88 & -464 & 237 & 50 & 427 \\ 153 & 978 & 13 & 576 & 421 \\ 975 & 136 & 960 & 281 & 533 \end{pmatrix}$$

$$M = \begin{pmatrix} 1 & & & & \\ -0.1244 & 1 & & & \\ 0.04684 & -0.4701 & 1 & & \\ 0.5527 & 1.2627 & -0.3044 & 1 & \\ 0.8466 & 0.0884 & 1.0443 & -0.7971 & 1 \end{pmatrix} \times B^*$$

Normes = (1490.3, 559.0, 681.2, 1220.2, 1501.2)

Vers la « size-reduction »

$$M = \begin{pmatrix} 782 & 550 & 745 & 851 & 167 \\ -17 & 318 & -258 & -327 & 194 \\ 88 & -464 & 237 & 50 & 427 \\ -612 & 110 & -474 & 52 & 60 \\ 975 & 136 & 960 & 281 & 533 \end{pmatrix}$$

$$M = \begin{pmatrix} 1 & & & & \\ -0.1244 & 1 & & & \\ 0.04684 & -0.4701 & 1 & & \\ -0.3227 & 0.2627 & -0.3044 & 1 & \\ 0.8466 & 0.0884 & 1.0443 & -0.7971 & 1 \end{pmatrix} \times B^*$$

Normes = (1490.3, 559.0, 681.2, 785.8, 1501.2)

Vers la « size-reduction »

$$M = \begin{pmatrix} 782 & 550 & 745 & 851 & 167 \\ -17 & 318 & -258 & -327 & 194 \\ 88 & -464 & 237 & 50 & 427 \\ -612 & 110 & -474 & 52 & 60 \\ -490 & -158 & -238 & -241 & -195 \end{pmatrix}$$

$$M = \begin{pmatrix} 1 & & & \\ -0.1244 & 1 & & \\ 0.04684 & -0.4701 & 1 & \\ -0.3227 & 0.2627 & -0.3044 & 1 \\ -0.3984 & -0.1787 & -0.2600 & 0.2028 \end{pmatrix} \times B^*$$

Normes = (1490.3, 559.0, 681.2, 785.8, 646.3)

La « size-reduction »

L'algorithme

procedure SizeReduce$(\mathbf{b}_1, \ldots, \mathbf{b}_d)$
  $\mu, B^* \leftarrow$ GramSchmidt$(\mathbf{b}_1, \ldots, \mathbf{b}_d)$
  for $i = 2$ to d **do**
    for $j = i - 1$ downto 1 **do**
      $\mathbf{b}_i \leftarrow \mathbf{b}_i - \lceil \mu_{ij} \rfloor \mathbf{b}_j$
      for $k = 1$ to j **do**
        $\mu_{ik} \leftarrow \mu_{ik} - \lceil \mu_{ij} \rfloor \mu_{jk}$
      end for
    end for
  end for
end procedure

- Temps polynomial !
  - Mais attention à la taille des rationnels...

La « size-reduction »

$$\begin{pmatrix} \mathbf{b}_1 \\ \vdots \\ \mathbf{b}_d \end{pmatrix} = \begin{pmatrix} 1 & & \\ \vdots & \ddots & \\ \mu_{d1} & \dots & 1 \end{pmatrix} \begin{pmatrix} \mathbf{b}_1^* \\ \vdots \\ \mathbf{b}_d^* \end{pmatrix}$$

Definition

La base $(\mathbf{b}_1, \dots, \mathbf{b}_d)$ d'un réseau est « size-reduced » (faiblement réduite ?) si $|\mu_{ij}| \leq \frac{1}{2}$ pour tout $1 \leq i < j \leq d$.

- Cela implique :

$$\|\mathbf{b}_i\|^2 \leq \|\mathbf{b}_i^*\|^2 + \frac{1}{4} \sum_{j=1}^{i-1} \|\mathbf{b}_j^*\|^2$$

- Donc ce serait bien que TOUS les $\mathbf{b}_i^*$ soient courts...

Propriétés des GSO

Theorem

$\mathrm{Vol}(\mathbf{b}_1, \ldots, \mathbf{b}_d) = \sqrt{\det(BB^t)} = \|\mathbf{b}_1^*\| \times \cdots \times \|\mathbf{b}_d^*\|$

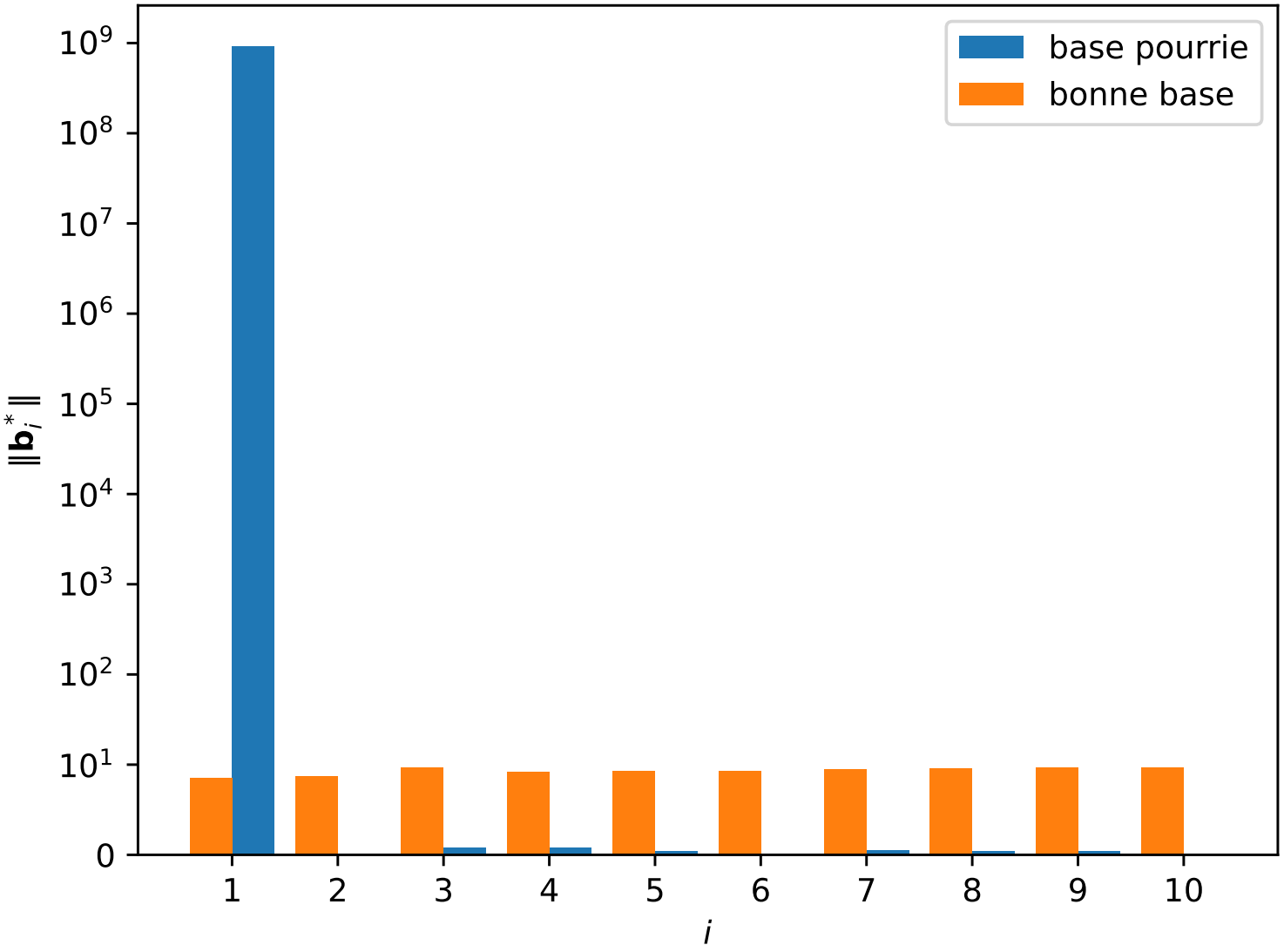
- Donc la situation « optimale » est $\|\mathbf{b}_i^*\| \approx (\mathrm{Vol}\, B)^{\frac{1}{d}}$

Preuve :

$$\begin{aligned}\det\left(BB^t\right) &= \det\left(\mu B^* B^{*t} \mu^t\right) \\ &= \underbrace{\det(\mu)}_{1} \times \det\left(B^* B^{*t}\right) \times \underbrace{\det\left(\mu^t\right)}_{1} \\ &= \det\left(B^* B^{*t}\right)\end{aligned}$$

$$B^* B^{*t} = \begin{pmatrix} \langle \mathbf{b}_1^*, \mathbf{b}_1^* \rangle & & \\ & \ddots & \\ & & \langle \mathbf{b}_d^*, \mathbf{b}_d^* \rangle \end{pmatrix} \rightsquigarrow \det\left(B^* B^{*t}\right) = \|\mathbf{b}_1^*\|^2 \times \cdots \times \|\mathbf{b}_d^*\|^2$$

Orthogonalisation de Gram-Schmidt : une mesure de qualité ?



Conclusion (partielle)

- Une base où les $\mathbf{b}_i^*$ sont de norme faible est une base intéressante
  - Size-reduction efficace (pour SVP)
  - Hyperplan le plus proche efficace (pour CVP)
- La size-reduction ne change pas les $\mathbf{b}_i^*$...
- Mais permuter les $\mathbf{b}_i$ change les $\mathbf{b}_i^*$...